

# Linux: Administración y Servicios

A.F.I.

Antonio Yáñez Izquierdo

Noviembre 2014

## Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Linux: generalidades

- ▶ Tradicionalmente la interacción con el sistema es a través del intérprete de comandos o *shell*
- ▶ Los comandos son estándar y su comportamiento, por lo general, difiere muy poco de un sistema a otro
- ▶ Algunos sistemas tienen conjuntos distintos de comandos de administración
- ▶ Aunque no es imprescindible para el funcionamiento del sistema, todos los sistemas unix actuales disponen de una interfaz gráfica. El aspecto de dicha interfaz es configurable y existen diversos entornos de escritorio opcionales (mate, gnome, wmaker, kde, xfce . . . )

# Linux: generalidades

- ▶ Sistema Operativo libre con licencia GNU
- ▶ Dos maneras de referirnos a el: por versión o por distribución
- ▶ **versión:** se suele referir a la versión del kernel (núcleo), 2.0, 2.4, 2.6, 3.0 . . . . Representa funcionalidades en el núcleo del sistema, dispositivos soportados . . .
- ▶ **distribución:** Ya que es un sistema libre y disponible, cualquiera puede hacer si propia *distribución* con las utilidades que considere convenientes, para un uso específico y con una filosofía particular: debian, gentoo, ubuntu, ubuntu studio, redhat, fedora, suse, slackware, galinux
- ▶ Código fuente disponible: puede ser auditado, revisado y modificado para adecuarlo distintas necesidades
- ▶ Totalmente independiente de la interfaz gráfica (incluso puede existir sin ella)

# Linux: arquitecturas soportadas

- ▶ Dado que es un sistema libre puede ser portado a multiples arquitecturas. Además al estar escrito en C se facilita la portabilidad
- ▶ Las arquitecturas mas usadas son PC 32 bits (i386) y PC 64 bits (amd64). Existen ports para muchas otras arquitecturas. Además de para las arquitecturas PC stá disponible (no todas las distribuciones, no enel mismo estadio de desaroollo) una versión actual para
  - ▶ alpha
  - ▶ sparc
  - ▶ IBM S/390
  - ▶ powerpc
  - ▶ MIPS (little endian y big endian)
  - ▶ ARM
  - ▶ intel itanium
  - ▶ HP PA-RISC

# Linux: configuración

- ▶ La configuración del sistema reside en archivos de texto, puede verse y modificarse
- ▶ Los archivos de configuración están típicamente en `/etc`
- ▶ Los archivos de texto son editables
- ▶ Suele haber una utilidad en modo texto para configurar la mayor parte de las cosas

# Linux: configuración

- ▶ También suele haber una utilidad para configurar la mayor parte de las cosas desde la interfaz gráfica
- ▶ por ejemplo, para añadir un usuario
  - ▶ añadir una línea a los archivos `/etc/passwd` y `/etc/shadow` y crear manualmente el directorio
  - ▶ usar el comando `adduser`
  - ▶ en *gnome*: `system— >administration— >users and groups`
- ▶ En los sistemas linux la inicialización del sistema, y muchas tareas de administración se realizan por medio de *scripts*
- ▶ Un *scripts* es un programa interpretado por el *shell* del sistema

# linux: usuarios

- ▶ Cada fichero en el sistema pertenece a un usuario y es de un grupo de usuarios
- ▶ Cada fichero en el sistema tiene unos permisos que indican que pueden hacer con él, su propietario, el grupo y el resto de usuarios
- ▶ Existe un usuario en el sistema *root* para el cual no rigen los permisos de los ficheros
- ▶ Cada usuario tiene privilegios sobre SUS ficheros



# linux: usuarios

- ▶ Solo el *root* tiene privilegios de *root*
- ▶ Los archivos del sistema son del *root*
- ▶ En linux existe el comando `sudo` que permite a algunos usuarios ejecutar comandos con privilegios de *root*
- ▶ Algunas instalaciones tienen usuarios que “ejecutan” diferentes servicios: `sshd`, `telnetd`, `www-data`
- ▶ Nunca se usa la máquina como *root*, salvo para tareas administrativas

# Administración: tareas

- ▶ Tareas del Administrador:
  - ▶ **Instalación:** Instalar el sistema y las aplicaciones
  - ▶ **Configuración:** Configurar el sistema y las aplicaciones instaladas de manera que
    - ▶ Las máquinas funcionen lo más correcta y eficientemente posible.
    - ▶ Los usuarios utilicen las máquinas de forma adecuada.
  - ▶ **Mantenimiento:** Mantener el sistema y las aplicaciones en perfecto estado de funcionamiento.
  - ▶ **Seguridad:** Garantizar la seguridad de los sistemas.

# Administración: tareas

- ▶ Automatización: Una prioridad en la administración es automatizar todas las tareas que sea posible. Ejemplos:
  - ▶ Backups.
  - ▶ Chequeo de espacio de disco.
- ▶ Monitorización: Comprobar el uso que se está haciendo de los sistema.
  - ▶ Uso de recursos.
  - ▶ Funcionamiento correcto de los servicios.
  - ▶ Comportamiento de los usuarios.

# Administración

- ▶ El sistema está preparado para que los usuarios normales no puedan dañar al sistema accidentalmente (ni intencionadamente).
- ▶ Un usuario normal no puede cambiar la configuración del sistema, instalar o desinstalar programas globalmente, o en general cualquier tarea que afecte a los demás usuarios.
  - ▶ Posibilidad de instalar programas *particularmente*.

# Administración

- ▶ Para realizar estas tareas existe una cuenta especial, la del administrador (*root*) que puede realizar cualquier cambio en el sistema.
- ▶ La cuenta de *root* debería utilizarse solo cuando sea necesario, por los problemas que puede causar un mal uso de la misma.
- ▶ Si al trabajar como usuario es necesario hacer algo como *root*, se puede utilizar *su* o *su -* para abrir una sesión como *root* en el terminal actual o utilizar el comando *sudo*

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Instalación de linux

conceptos generales

discos y particiones

gestores de arranque

Actividades

# Instalación de linux

- ▶ El uso mas común de linux es tenerlo *instalado* en el sistema,
  - ▶ hay algunas distribuciones “*live*” que no requieren instalación pero usualmente tiene limitaciones en cuanto a lo que podemos hacer y/o añadir
- ▶ *instalación* es el proceso mediante el cual colocamos el S.O. en las unidades de almacenamiento de nuestro sistema y lo ejecutamos directamente desde ahí



# Instalación de linux

- ▶ el proceso de instalación usualmente incluye los siguientes pasos
  - a arrancar el sistema desde el medio de instalación
  - b escribir los archivos del sistema operativo en la unidad de almacenamiento deseado
  - c realizar la configuración necesaria para que el S.O. se cargue desde el medio de almacenamiento especificado
  - d reiniciar

# Instalación de linux

- a para arrancar del medio de instalación necesitamos: el medio de instalación
  - ▶ podemos obtenerlo ya preparado
  - ▶ tenemos que prepararlo nosotros
- b escribir los archivos del S.O. a nuestra unidad de almacenamiento suele implicar el *particionado* de dicha unidad
- c hacer que el sistema operativo arranque del medio de almacenamiento especificado implica instalar y configurar el gestor de arranque (*boot loader*)

# medios de instalación

- ▶ los medios de instalación mas empleados hoy en dia son
  - ▶ dispositivos CD/DVD
  - ▶ dispositivos usb
  - ▶ tarjetas de red (Network Interface Cards)

# iniciar desde la red

- ▶ los sistemas modernos permiten ser iniciados desde la red
- ▶ iniciar desde la red requiere la configuración de un *servidor de arranque*, que proporciona el arranque a nuestro sistema
- ▶ uno de los modos de instalación mas usual es lo que se llama instalación en red (*network installation*) y que incluye,
  - ▶ arranque desde un dispositivo CD/DVD o usb
  - ▶ hacer una configuración básica de la red
  - ▶ obtener por la red los ficheros del S.O.

# imágenes de instalación

- ▶ las imágenes de instalación mas comunes son
  - ▶ imágenes ISO (para grabar en un CD/DVD)
  - ▶ imágenes especiales para copiar en una memoria usb
- ▶ si usamos un software de virtualización (p.e. virtualbox), podemos instalar directamente desde las imágenes ISO

# Instalación de linux

conceptos generales

**discos y particiones**

gestores de arranque

Actividades

# discos

- ▶ los discos son el medio mas usual en donde instalar los S.O.
- ▶ hoy en día todos los discos usan LBA (Logical Block Addressing) en lugar de la antigua interfaz CHS, aunque todavia informan parámetros (falsos) CHS
- ▶ tambien informan de un tamaño de sector de 512 bytes aunque internamente usen sectores 4096 bytes

# disks

- ▶ usamos discos para crear sistemas de ficheros en ellos
- ▶ varios sistemas de ficheros pueden existir en un disco en lo que usualmente llamamos *particiones*
- ▶ varias *particiones* pueden ser combinadas en un sistema de ficheros usando software LVM (**L**ogical **V**olume **M**anagement)



# particiones

- ▶ un disco suele estar dividido en *particiones*
- ▶ se crean sistemas de ficheros en las *particiones*, normalmente un sistema de ficheros en cada *partición* aunque usando LVM podemos combinar varias particiones en un sistema de ficheros (windows designa cada sistema de ficheros con una letra, por ejemplo **C:**)
- ▶ distintos sistemas operativos pueden coexistir en una misma máquina en distintas particiones

# tablas de particiones

- ▶ cada disco tiene una tabla, usualmente en su primer bloque, que define las particiones en ese disco
- ▶ hay distintos formatos para esa tabla
  - ▶ particiones MBR
  - ▶ GUID Partition Table (GPT)
  - ▶ BSD disklabel
  - ▶ Solaris VTOC label
  - ▶ Amiga Rigid Disk Block (RDB)
  - ▶ Apple Partition Map (APM)
  - ▶ ...

# particiones MBR

- ▶ la tabla de particiones está en el primer sector del disco
- ▶ es la más extendida en la arquitectura PC
- ▶ usada principalmente por sistemas Windows y linux
- ▶ en un disco pueden definirse hasta 4 particiones, llamadas particiones primarias

# particiones MBR

- ▶ una de las particiones puede definirse como *partición extendida*
- ▶ esta particion puede subdividirse en lo que se llama *unidades lógicas* o *particiones lógicas*
- ▶ el primer sector de dicha partición, llamado EBR (Extended Boot Record), tiene el mismo formato que la tabla de particiones MBR excepto que
  - ▶ solo se usan las dos primeras entradas
  - ▶ si se necesitan mas particiones, una de ellas se define como *extendida*, pudiendo por tanto crearse "*infinitas*" particiones

# particiones GPT (GUID Partition Table)

- ▶ definida como parte del estándar EFI (Extensible Firmware Interface)
- ▶ tambien denominada tabla de particiones EFI
- ▶ MBR usa 32 bits para LBA, por tanto no soporta medios de mas de  $2^{32}$  sectores (2TB)
- ▶ GPT usa 64 bits para LBA, lo que limita el tamaño de las particiones a  $2^{64} - 1$  sectores
- ▶ soportada por la mayor parte de los S.O: modernos (linux entre ellos)

# tabla de particiones GPT

- ▶ existen dos copias de la GPT, la *GPT primaria* al principio del disco, y la *GPT secundaria* al final
- ▶ el primer sector sigue teniendo una tabla MBR llamada *protective MBR* que permite que el disco arranque con una BIOS tradicional
- ▶ El tamaño mínimo de la GPT es de 16Kbytes, lo que hace que tenga un mínimo de 128 entradas

# Instalación de linux

conceptos generales  
discos y particiones  
gestores de arranque  
Actividades

# cargadores del S.O: linux

- ▶ linux dispone de varios cargadores
- ▶ los mas usados son: `lilo`, `silos` (para Sparc), y `grub` que es el estándar hoy en día
- ▶ `lilo` puede cargar otros S.O. haciendo *chain* a su cargador
- ▶ `grub` puede cargar otros S.O. directamente o haciendo *chain* a su cargador



# lilo

- ▶ **era** el cargador estándar de linux
- ▶ puede hacer *chain* a otros cargadores
- ▶ se configura en `/etc/lilo.conf`
- ▶ no entiende los sistemas de ficheros, por lo tanto no puede leer su fichero de configuración al arrancar. Debemos ejecutar después de modificar su configuración `/sbin/lilo`
- ▶ puede instalarse tanto en el MBR como en el principio de la partición
- ▶ para sistemas EFI existe `elilo`

# ejemplo de configuración de lilo

```
boot=/dev/sda1
root=/dev/sda1
image=/boot/vmlinuz-2.6.38-2-amd64
    label="linux 2.6.38"
    initrd=/boot/initrd.img-2.6.38-2-amd64
    read-only
other=/dev/sda2
    label="otroSistema"
```

# grub

- ▶ es el cargador por defecto en la mayoría de distribuciones de linux
- ▶ dos versiones
  - ▶ versión *antigua* de *grub*: muchas veces denominada *grub legacy*
  - ▶ versión *moderna* de *grub*: *grub 2*

# grub

- ▶ *grub* lee su configuración al arrancar y puede cargar directamente otros S.O.
- ▶ puede instalarse tanto en el MBR como en el principio de la partición
- ▶ el fichero de configuración
  - ▶ /boot/grub/menu.lst para *grub legacy*
  - ▶ /boot/grub/grub.cfg para *grub2*

# grub legacy

- ▶ la configuración reside en `/boot/grub/menu.lst`
- ▶ se instala con `grub-install`
- ▶ tras cambiar su archivo de configuración no hay que hacer nada más
- ▶ ejemplo de configuración

```
title linux with kernel 2.4.7-10
    root (hd0,1)
    kernel /boot/vmlinuz-2.4.7-10 ro root=/dev/hda5
    initrd /boot/initrd-2.4.7-10.img
title otrosSistema
    rootnoverify (hd0,2)
    chainloader +1
```

# grub2

- ▶ la configuración reside en `/boot/grub/grub.cfg`
- ▶ este fichero no debe modificarse pues se genera automáticamente con *update-grub2* (o *grub2-mkconfig* según la version)
- ▶ configuración manual del *grub2* debe hacerse en los ficheros en `/etc/grub.d` y luego ejecutar *update-grub2*
  - ▶ los contenidos de `40_custom` and `41_custom` son copiados en `/boot/grub/grub.cfg`. En algunas versiones puede usarse tambien el archivo `custom.cfg`, en el directorio principal del *grub*

# grub2

## ► ejemplo del fichero 40\_custom

```
#!/bin/sh
exec tail -n +3 $0
# This file provides an easy way to add custom menu entries.  Simply type the
# menu entries you want to add after this comment.  Be careful not to change
# the 'exec tail' line above.

}
menuentry "otroSistema" {
    set root=(hd0,msdos2)
    chainloader +1
}
```

# Instalación de linux

conceptos generales  
discos y particiones  
gestores de arranque  
Actividades



# Actividades

- 1 Importar en Virtualbox la máquina virtual suministrada Cursolinux2014. (si no esta hecho)
- 2 Instalar fedora linux en dicha máquina virtual conservando el S.O. que hay en ella
  - ▶ Instalarla en varias particiones distintas sin utilizar LVM
  - ▶ Como gestor de arranque instalar *grub* en el Master Boot Record
- 3 Importar en Virtualbox la máquina virtual suministrada Cursolinux2014B.(si no esta hecho)
- 4 Instalar debian linux en dicha máquina virtual conservando el S.O. que hay en ella
  - ▶ debe instalarse en UNA SOLA partición primaria
  - ▶ durante el proceso de instalación instalar *grub* en dicha partición
  - ▶ sustituir *grub* por *lilo* y comprobar que funciona

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Shell Scripting

## Variables y estructuras de control

Actividades 1

Manipulación de texto

Actividades 2

# Shell Scripting

- ▶ Se suele denominar *script* un fichero de texto que contiene comandos entendibles por un intérprete: perl (perlscrip), python .....
- ▶ Los shells de unix (bash, csh, ksh, sh, tcsh ...) son un caso particular de intérpretes
- ▶ Un *shell script* es una serie de comandos escritos en un fichero de texto plano (sin formato) para poder ejecutar tareas repetitivas que normalmente haríamos en un terminal.
- ▶ Se pueden utilizar también para extraer información de textos.
- ▶ En un sistema linux se utilizan en muchas partes del sistema, por ejemplo: scripts de inicialización. programas de instalación..
- ▶ El shell de unix puede considerarse como (y de hecho lo es) un lenguaje de programación.

# Shell Scripting : Estructura

- ▶ La estructura de un script es la siguiente

```
#!/bin/sh
```

```
comando1
```

```
comando2
```

```
comando3 ; comando4
```

```
....
```

```
# Esto es un comentario
```

- ▶ La primera linea indica que programa (shell) lo va a interpretar, tras los símbolos `#!`
- ▶ Los comandos se separan bien utilizando `;` o empezando en una nueva linea.
- ▶ Los comentarios se indican con `#`, y abarcan hasta el final de la linea.
- ▶ Para poder ejecutarlo hay que darle permisos de ejecución.

# Shell Scripting: variables

## ► Comandos Usuales:

- Imprimir Texto: `echo [-n] "texto"`. Si se incluye la opción `-n` no se crea una nueva línea tras terminar el comando.
- Variables:
  - No se declaran, y no tienen tipos.
  - La variable se identifica por su nombre (Ej *prueba*), pero su valor se accede anteponiéndole un `$`. (Ej *\$prueba*).
  - las mayúsculas son significativas para el nombre de las variables.
  - El sistema define una serie de variables con valores útiles, (variables de entorno) como por ejemplo *\$HOME*, que contiene el directorio home del usuario. Ej: `/home/prueba`
  - Para asignarle un valor a una variable se usa el `=`, pero no puede haber espacios entre la variable, el `=` y la expresión a asignar.

# Shell Scripting: let

- ▶ Para calcular el resultado de alguna operación se usar let.
- ▶ Las operaciones disponibles son:
  - ▶ Aritméticas básicas: +, -, \*, /, %.
  - ▶ Incrementos: id++, ++id.
  - ▶ Decrementos: id--, --id.
  - ▶ Exponenciación: \*\*.
  - ▶ Deplazamientos: <<, >>.
  - ▶ Comparaciones: <, >, <=, >=.
  - ▶ Igualdad: ==, !=.
  - ▶ Operadores lógicos: &&, ||.
  - ▶ Operaciones sobre bits: &, ^, |.

- ▶ Ejemplo:

```
#!/bin/sh
let x=4+5*3
echo $x
```

# Shell Scripting: `expr`

- ▶ También se puede utilizar el comando **`expr`**.
- ▶ **`expr EXPRESION`** imprime en la salida estándar el resultado de `EXPRESION`. `EXPRESION` puede ser
  - ▶ `arg1 | arg2` `arg1` si distinto de 0 o nulo, en otro caso `arg2`.
  - ▶ `arg1 & arg2` `arg1` si es 0 o nulo, en otro caso `arg2`.
  - ▶ `arg1 < arg2` 1 si `arg1 < arg2` 0 en otro caso. Todos los operadores relacionales (`<`, `<=`, `>`, `>=`, `=`, `!=`) opera tanto sobre enteros como sobre cadenas.
  - ▶ `arg1 + arg2`. Suma aritmética. (Otros operadores aritméticos: `-`, `*`, `/`, `%`).

- ▶ Ejemplo:

```
#!/bin/sh
x=`expr 4 + 5`
echo $x
```



# Shell Scripting: comillas

- ▶ El texto se puede rodear con 3 tipos de comillas, con interpretación diferente:
  - ▶ 'Texto': (comilla simple, a la derecha del 0 en el teclado español) Anula el significado especial de la mayoría de los caracteres. Por ejemplo, no se referencian las variables.
  - ▶ "Texto": Al contrario que la anterior, los caracteres se interpretan.
  - ▶ `Texto`: (comilla invertida, a la derecha de la p en el teclado español) Se entiende que texto es un programa con parámetros, y se toma el resultado de ejecutarlo.

```
FILE=/home/yo/p1.c
```

```
PRU='wc -l $FILE'#'wc -l $FILE'
```

```
PRU="wc -l $FILE" #'wc -l /home/yo/p1.c'
```

```
PRU=`wc -l $FILE`#'457'
```

# Shell Scripting: parámetros

- ▶ Para leer datos introducidos a través del teclado se usa `read`. Ejemplo:  
`read fname`
- ▶ Argumentos: Son los parámetros que se le pasan al comando al ejecutarlo. Se pueden acceder desde el script mediante las variables `$1...$9`.
- ▶ `$#` tiene el número de argumentos que se le han pasado al comando.
- ▶ `$$` el pid del shell.
- ▶ `$?` valor devuelto por último comando.
- ▶ `$IFS` lista de caracteres separadores.

# Shell Scripting: redirección

- ▶ La entrada y salida estándares de un comando se pueden redirigir a ficheros. Existen tres formas de redirección:
  - ▶ `>` redirecciona la salida del comando a un fichero.
  - ▶ `<` obtiene la entrada estándar de un fichero.
  - ▶ `>>` concatena la salida del comando a un fichero (Sin eliminar los contenidos anteriores).
- ▶ `> &fich` redirecciona el error estándar a un fichero *fich*. (dependiente del shell); `2 > fich` también lo hace (sin espacio entre el 2 y `>`)
- ▶ Para comunicar un programa con otro se utilizan pipes. Un pipe redirige la salida estándar de un comando a otro comando. Ejemplo:  

```
ls | more; cat *.c | wc -l
```

# Shell Scripting: if

## ► If:

- Permite la ejecución de partes de código condicionadas al cumplimiento de una determinada condición. Sigue la siguiente sintaxis:

```
if condicion; then comando1; comando2; ...  
else comando4; comando5 fi
```

```
if condicion then ....;  
elif condicion then .... ;  
else .... fi
```

# Shell Scripting: case

- ▶ case:

- ▶ Permite la ejecución de partes de código dependiendo del valor de una variable.

```
case palabra in
patron1) comandos ;;
patron2) comandos ;;
....
esac
```

- ▶ case compara la palabra con los patrones empezando por arriba y ejecuta los comandos del primer patrón que coincide.

# Shell Scripting: case

## ► ejemplos del case

```
case $# in
0) ..... ;;      #caso sin argumentos
1) ..... ;;      #con un argumento
*) ..... ;;      #en otro caso
esac
```

```
case $1
start) ..... ;;
stop) ..... ;;
*)
    echo uso {start|stop}
    exit 0
;;
esac
```

# Shell Scripting: test

- ▶ Test o `[]`: Se utiliza para comprobar si determinadas condiciones son ciertas. La sintaxis es la siguiente:

`test EXPRESION ó [ EXPRESION ]`

- ▶ EXPRESION puede ser
  - ▶ `( EXP )` EXP es cierta
  - ▶ `! EXP` EXP es falsa
  - ▶ `EXP1 -a EXP2` son ciertas EXP1 y EXP2
  - ▶ `EXP1 -o EXP2` una dc EXP1 o EXP2 es cierta
- ▶ EXP puede involucrar números, cadenas o ficheros.

# Shell Scripting: test

- ▶ Algunas de las condiciones sobre números que podemos comprobar con test (A y B representan enteros):
  - ▶ A -eq B : Igualdad numérica.
  - ▶ A -ne B : Desigualdad numérica.
  - ▶ A -gt B : Mayor que.
  - ▶ A -lt B : Menor que.
  - ▶ A -ge B : Mayor o igual que.
  - ▶ A -le B : Menor o igual que.



# Shell Scripting: test

- ▶ Condiciones sobre cadenas ( $S$  y  $T$  representan cadenas):
  - ▶  $S = T$  : Igualdad. (con espacios antes y después del  $=$ )
  - ▶  $S \neq T$  : Desigualdad.
  - ▶  $-n S$  : Comprueba si no contiene el valor nulo y existe.
  - ▶  $-z S$  : La longitud de  $S$  es 0

# Shell Scripting: test

## ► Condiciones sobre ficheros:

- `f1 -ef f2` : tienen mismo dispositivo e inodo
- `f1 -nt f2` : `f1` es mas reciente que `f2` (`-ot` mas antiguo)
- `-s fichero` : El tamaño del fichero es mayor que 0.
- `-e fichero` : El fichero existe.
- `-f fichero` : El tipo de fichero debe ser un fichero normal, no un directorio o un dispositivo.
- `-d fichero` : El fichero debe ser un directorio.
- `-b fichero` : El fichero debe ser un dispositivo de bloque.
- `-c fichero` : El fichero debe ser un dispositivo de caracter.
- `-h fichero` : El fichero debe ser un enlace simbólico.
- `-r fichero` : Se puede leer el fichero.
- `-w fichero` : Se puede escribir.
- `-x fichero` : Se puede ejecutar.
- `-u fichero` : Es un fichero setuid.
- `-g fichero` : Es un fichero setgid

# Shell Scripting: bucles

- ▶ Los dos bucles básicos son `for` y `while`:
  - ▶ El bucle `for` permite la repetición de instrucciones un cierto número de veces.  

```
for variable in lista; do comando1;  
comando2; done
```
  - ▶ Para generar la lista de valores sobre los que se itera es cómodo usar *seq*.  

```
for i in `seq 1 100`; do echo $i; done
```
  - ▶ El bucle `while` ejecuta instrucciones mientras se cumpla una condición.  

```
while condicion do comando1;  
comando2; done
```

# Shell Scripting

Variables y estructuras de control

Actividades 1

Manipulación de texto

Actividades 2

# Shell Scripting. Actividades

- ▶ Crear un script, *factorial.sh* que calcule la función factorial de un número que le pasemos por la línea de comandos.
- ▶ Crear un script, *hosts.sh*, que genere un fichero de hosts para máquinas consecutivas es decir pc01, pc02, pc03, con ips 192.168.1.1, 192.168.1.2... El fichero de hosts tiene el siguiente formato:

```
direccion_ip Nombre_maquina
```

El primer argumento, en caso de suministrarse, será el nombre de los equipos (por defecto PC); el segundo la última ip (por defecto 254) y el tercero la red (por defecto 192.168.0)

# Shell Scripting

Variables y estructuras de control

Actividades 1

**Manipulación de texto**

Actividades 2

# Shell Scripting: cut y tr

- ▶ Desde un script del shell puede llamarse a cualquier programa externo. Los mas usuales para procesado de texto son cut, tr, sed, grep, paste, cat, echo ...
  - ▶ cut: permite obtener determinados partes de un texto.  
`cut -f campo -d delimitador.`  
Ej:  
`$cut -f 1,4 -d " "`
  - ▶ tr: Realiza cambios o eliminaciones de caracteres.  
`tr [CAR1] [CAR2].`  
Ej: `tr [a-z] [A-Z] < fichero`  
`tr -s "\t " # Eliminación de Repetidos.`

# Shell Scripting: sed

- ▶ sed: Es un editor de líneas no interactivo.
  - ▶ Lee la entrada estándar (o el fichero que se le pasa como parámetro)
  - ▶ Realiza una serie de operaciones sobre algunas líneas
  - ▶ Escribe el resultado por la salida estándar.
- ▶ sed determina sobre qué líneas debe actuar utilizando un rango o un patrón que se le indica. Si no le especifica ningún rango, por defecto actúa sobre todas las líneas.
- ▶ sed va imprimiendo las líneas según las va procesando. Si queremos que solo imprima las seleccionadas debemos hacer `sed -n`



# Shell Scripting: sed

## ► Selección de líneas

- `/regex/`: líneas que contienen la expresión regular `regex`
- `\cregexp`: líneas que contienen la expresión regular `regex` precedida y terminada por el carácter `c`
- `n~M`: a partir de la línea `n` cada `M` líneas
- `li,+M`: la línea `li` y las `M` siguientes. `li` puede representar un número o una expresión.
- `li,~M`: la línea `li` y las siguientes hasta un múltiplo de `M`.

# Shell Scripting: sed

- Imprimir líneas: [rango]p.

```
$sed -n 3,+4p <fichero
```

```
#imprime líneas 3,4,5,6,7
```

```
$sed -n /que/p
```

```
#las que contienen 'que'
```

```
$sed -n /hola/,~3p < fichero
```

```
#desde la línea que contiene hola
```

```
#a la primera que es múltiplo de 3
```

- Borrar líneas: [rango]d.

```
$sed 3,+4d <fichero
```

```
#elimina líneas 3,4,5,6,7
```

```
$sed /que/d
```

```
#las que contienen 'que'
```

```
$sed /hola/,~3d < fichero
```

```
#desde la línea que contiene hola
```

```
#a la primera que es múltiplo de 3
```

# Shell Scripting: sed

- ▶ Sustituir cadenas: s/pattern1/pattern2/  
`$sed s/hola/HOLA/ < fichero`
- ▶ Sustituir caracteres y/caracteres1/caracteres2/  
`$sed y/afk/JKL/ < fichero`

# Shell Scripting: grep

- ▶ grep: Realiza búsquedas en base a expresiones regulares.

```
cat file.txt | grep "[0-9]\+\.[0-9]*"
```

# Shell scripting: head y tail

- ▶ hay más utilidades que seleccionan partes de un fichero
- ▶ **head** selecciona la primera parte de un fichero (líneas, caracteres...)  
`$ head -15 < fil # primeras 5 lineas del fichero fil`
- ▶ **tail** selecciona la última parte de un fichero (líneas, caracteres...)  
`$ tail -20 < fil # ultimas 20 lineas del fichero fil`
- ▶ **wc** cuenta líneas, palabras y caracteres en un fichero

# Shell scripting: funciones

- ▶ Los shells modernos permiten incluir funciones en sus scripts

```
nombre_function (){  
    sentencias ejecutables  
    ...  
}
```

- ▶ una función puede recibir argumentos. Dentro de la función podemos acceder a sus argumentos con las variables \$1, \$2 ...
- ▶ para invocar una función se hace igual que invocaríamos un programa con sus argumentos
- ▶ para devolver un valor, la función puede hacer **echo** del valor con lo que puede ser asignado a una variable con las comillas inversas ' o con \$()

# Shell scripting: ejemplo de funciones

```
#!/bin/bash
suma (){

    let result=$1+$2
    echo $result

}

VAR1='suma 2 2'
VAR2=$(suma 2 3)

echo VAR1=$VAR1' ' VAR2=$VAR2
```

# Shell Scripting

Variables y estructuras de control

Actividades 1

Manipulación de texto

Actividades 2



# Shell Scripting. Actividades

- ▶ Utilizando las utilidades anteriores, crear un script, *procesos.sh* que liste los procesos que esté ejecutando el sistema que ocupen más de una cierta cantidad de memoria que se le indica como parámetro.
- ▶ Crear un script, *ficheros.sh* que liste todos los ficheros en un directorio que ocupen más de un determinado espacio que se pasa como parámetro. Utilizar un argumento opcional para indicar si deben ser eliminados o comprimidos.
- ▶ Crear un script *backup.sh* que haga una copia de seguridad de los ficheros de los usuarios que tienen cuenta en la máquina. Dichas copias de seguridad deben residir en el directorio `/backup/fecha_de_la_copia` (que si no existe debe ser creado) y solo podrán ser accedidas por su propietario). Si ya existe copia de seguridad de ese día el script NO HARA NADA.

Introducción

Instalación de linux

Shell Scripting

**Tareas Planificadas**

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Tareas Planificadas

cron

crontab

el comando at

Actividades

# Tareas Planificadas: Cron

- ▶ Las tareas que deben ejecutarse regularmente las gestiona cron.
- ▶ Fichero `/etc/crontab`: este fichero está formado por una serie de líneas. Cada línea indica una determinada tarea, así como la periodicidad con la que debe ejecutarse.
- ▶ La línea contiene los siguientes campos:
  - ▶ Minuto (0-59)
  - ▶ Hora (0-23)
  - ▶ Día del Mes (1-31)
  - ▶ Mes
  - ▶ Día de la Semana (0-7)
  - ▶ Usuario
  - ▶ Comando
- ▶ Un `*` en cualquiera de los campos indica que cualquier valor es aceptable.

# Tareas Planificadas: Cron

- ▶ Se pueden especificar listas de dos formas: 1,2,3 ó 1-3.
- ▶ Además se puede especificar escalonados de tiempo con /step.
- ▶ Ejemplos:

```
# m h dom mon dow user  command
17 * * * * * root run-parts --report /etc/cron.hourly # A las horas y 17
*/17 * * * * * root run-parts --report /etc/cron.hourly # Cada 17 minutos
* * */2 * * * root /opt/tivoli/bin/tsm -d /home # Cada 2 días
* * 1-6/2 * * * root /sbin/e2fsck /dev/hda # El 1, 3 y 5 de cada mes
```

- ▶ Algunas distribuciones crean directorios en /etc, como /etc/cron.hourly, /etc/cron.daily... que permiten ejecutar tareas regularmente poniendo un script dentro de ellos sin necesidad de modificar crontab.

# Tareas Planificadas

cron

**crontab**

el comando at

Actividades

# crontab

- ▶ Además de una configuración global para el sistema, cada usuario puede tener sus tareas planificadas. Estas tablas se guardan en `/var/spool/cron`.
- ▶ El usuario puede modificar su tabla de tareas utilizando el comando `crontab -e`. Este comando ejecutará el editor indicado por la variable de entorno `$EDITOR`.
- ▶ Otros comandos:
  - ▶ `crontab -l`: Lista la tabla cron del usuario.
  - ▶ `crontab -r`: Elimina la tabla cron del usuario.

# Autorización/Denegación Usuarios

- ▶ El administrador puede especificar que usuarios tienen permiso para utilizar crontab.
- ▶ Existen dos ficheros para ello:
  - ▶ `/etc/cron.allow`: Si este fichero existe, solo los usuarios explícitamente listados en el podrán especificar tareas planificadas.
  - ▶ `/etc/cron.deny`: Si el fichero anterior no existe y este sí, entonces podrán especificar tareas todos los usuarios no listados en él.
- ▶ Si no existe ninguno de los dos, se entenderá que todos los usuarios pueden especificar tareas.



# Tareas Planificadas

cron

crontab

el comando at

Actividades

# el comando at

- ▶ el comando `at` (o *batch*) permite a un usuario remitir un trabajo para una ejecución posterior
- ▶ los trabajos se envían para ser ejecutados **una vez**
- ▶ **cron** usa los ficheros `at.allow` y `at.deny` para manejar la autorización, de manera análoga a como utiliza los ficheros `/etc/cron.allow` y `/etc/cron.deny` para el comando `crontab`

# Tareas Planificadas

cron

crontab

el comando at

Actividades

# Actividades

- ▶ Utilizar el comando `crontab` para ejecutar cada minuto un comando que le mande un mail a root diciendole la hora.
- ▶ Denegar a un usuario el acceso al `crontab` de dos maneras distintas. Comprobar que funciona

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Arranque de Servicios

## Proceso de arranque

runlevels

linux debian: `insserv`

fedora linux

Actividades

# Linux: proceso de arranque

- ▶ Después de realizar el Power On Self Test, la BIOS lee el primer sector del disco que esté configurado como de arranque y le transfiere el control
- ▶ El primer sector del disco (Master Boot Record MBR) lee el primer sector de la partición marcada como activa y le transfiere el control
- ▶ En el primer sector de la partición de linux (o en el Master Boot Record) reside el gestor de arranque. Los dos gestores de arranque más usuales son lilo y grub.
  - ▶ grub lee su configuración al arrancar de `/boot/grub/menu.lst` o `/boot/grub/grub.cfg` (a partir de versión 2)
  - ▶ lilo se configura al instalarlo y su configuración está en `/etc/lilo.conf`

# Linux: proceso de arranque

- ▶ El gestor de arranque, tras presentar un menú, carga el kernel que se le indique en memoria (típicamente está el directorio `/boot`) usando si es necesario la imagen *ramdisk*
- ▶ El kernel tras inicializar el sistema (vectores de interrupción, interfaz de llamadas ...) crea el primer proceso de usuario en el sistema `init`
- ▶ `init` Comprueba en el fichero `/etc/inittab` a que *runlevel* hay que llevar el sistema y tras invocar a los *scripts* que hay en `/etc/rcS.d` invoca los de `/etc/rcN.d` que irán arrancando los servicios disponibles en esa instalación



# Linux: proceso de arranque

- ▶ Cuando el sistema se apaga, se realiza un proceso similar para detener al sistema de forma ordenada.
- ▶ Sin embargo, durante el funcionamiento del sistema, es posible que deseemos tener determinados servicios ejecutandose y otros no. Debido a eso, el sistema tiene más de una configuración de funcionamiento normal.

# Arranque de Servicios

Proceso de arranque

runlevels

linux debian: `insserv`

fedora linux

Actividades

# El sistema de inicio: Runlevels

- ▶ Los runlevels son configuraciones de funcionamiento del sistema. Al entrar en cualquiera de los runlevels, se inician o paran determinados servicios de la máquina.
- ▶ Por defecto están definidos 7 runlevels (los significados dependen de la distribución).
  - ▶ 0: Parada de la máquina.
  - ▶ 1: Modo monousuario.
  - ▶ 2: Varios Usuarios sin entorno gráfico.
  - ▶ 3: Varios Usuarios con entorno gráfico.
  - ▶ 4: Disponible.
  - ▶ 5: Disponible.
  - ▶ 6: Reiniciado de la máquina.
- ▶ Para cambiar de runlevel `telinit`
- ▶ Para saber en que runlevel estamos `runlevel`

# El sistema de inicio: Runlevels

- ▶ Existe un directorio común para todos los runlevels donde se encuentran scripts que sirven para iniciar y detener los distintos servicios de la máquina. Dependiendo de la distribución puede estar en `/etc/init.d`.
- ▶ Estos scripts aceptan una serie de parametros estándar (start, stop, restart), que permiten indicar la acción que se desea realizar. Esto también permite iniciarlos o pararlos a mano.
- ▶ Para cada runlevel hay un directorio `rcN.d` (`rc0.d`, `rc1.d`, `rc2.d...`), bien en `/etc/rcN.d`, o en `/etc/rc.d/rcN.d`. Este directorio contiene enlaces a los scripts del directorio `init.d`. Estos enlaces tienen la siguiente forma

`S00exim`

`K20ssh`

# El sistema de inicio: Runlevels

- ▶ La primera letra (S o K) indica si el servicio debe ser iniciado en ese runlevel (start) o debe ser parado (kill).
- ▶ El número sirve para ordenar el inicio y parada de los servicios. Se inician o paran antes los que tienen un número menor.
- ▶ Si queremos realizar determinadas acciones al inicio del sistema, solo hay que crear un script, y situarlo en el runlevel adecuado.
- ▶ Es posible cambiar de runlevel mediante la aplicación telinit, pasandole como parámetro el runlevel al que queremos cambiar. Por ejemplo, telinit 1 pasaría el sistema a modo monousuario.

# Arranque de Servicios

Proceso de arranque

runlevels

linux debian: insserv

fedora linux

Actividades

# El sistema de inicio: insserv

- ▶ Proporciona un control mas sofisticado de las dependencias entre los scripts de inicialización
- ▶ Permite la ejecución de *scripts* en paralelo
- ▶ Introducido en debian *squeeze*
- ▶ Genera automaticamente los enlaces a los *scripts* en `/etc/init.d` desde los `/etc/rcN.d` adecuados
- ▶ Ejemplo: para crear los enlaces a *anacron* en los runlevels por defecto  

```
# insserv -d anacron
```
- ▶ Ejemplo: para eliminar los enlaces y que no se ejecute el *anacron*  

```
# insserv -r anacron
```

# El sistema de inicio: insserv

- Para usar el sistema insserv con un script es necesario que el *script* tenga una cabecera determinada

```
### BEGIN INIT INFO
# Provides:          boot_facility_1 [ boot_facility_2 ...]
# Required-Start:    boot_facility_1 [ boot_facility_2 ...]
# Required-Stop:     boot_facility_1 [ boot_facility_2 ...]
# Should-Start:      boot_facility_1 [ boot_facility_2 ...]
# Should-Stop:       boot_facility_1 [ boot_facility_2 ...]
# X-Start-Before:    boot_facility_1 [ boot_facility_2 ...]
# X-Stop-After:      boot_facility_1 [ boot_facility_2 ...]
# Default-Start:     run_level_1 [ run_level_2 ...]
# Default-Stop:      run_level_1 [ run_level_2 ...]
# X-Interactive:     true
# Short-Description: single_line_description
# Description:       multiline_description
### END INIT INFO
```

- Las descripciones de los boot.facilities pueden verse en /etc/insserv.conf o en los ficheros del directorio en /etc/insserv.conf.d/
- Para especificar dependencias tambien puede usarse \$all o \$null



# El sistema de inicio: insserv

- ▶ Si queremos incluir un script que se ejecute al principio lo mas comodo es incluirlo en `/etc/rc.local`
- ▶ Ejemplo de cabecera de *script* sencillo que se inicia en los runlevels 2,3,4,5 y S y se para en 0, y 6

```
### BEGIN INIT INFO
# Provides:          cortafuegos
# Required-Start:    $network
# Required-Stop:
# Default-Start:     2 3 4 5 S
# Default-Stop:      0 6
# Short-Description: Inicia o para el cortafuegos
# Description:       Rechaza conexiones de fuera del departamento
#                    que venga a un puerto distinto de 80
### END INIT INFO
```

# debian

- ▶ la manera más sencilla de que se ejecute un script al inicio es incluirlo en `/etc/rc.local`
- ▶ también podemos usar el comando `update-rc.d` para habilitar o deshabilitar servicios
- ▶ se puede forzar el comportamiento antiguo (orden manual de los scripts) creando el fichero `.legacy-bootordering` en el directorio `/etc/init.d`

```
# touch /etc/init.d/.legacy-bootordering
```

# Arranque de Servicios

Proceso de arranque

runlevels

linux debian: `insserv`

fedora linux

Actividades

# fedora systemctl

- ▶ los servicios del sistema en fedora se controlan mediante el comando `systemctl`
- ▶ el comando `systemctl` nos permite iniciar, parar, reiniciar ... lo que el llama *units*
- ▶ una *unit* puede ser de tipo *service* (proceso), *socket* (conexión), *device* (dispositivo), *mount* (punto de montaje) ...
- ▶ ejemplos
  - ▶ para iniciar telnet

```
# systemctl start telnet.socket
```
  - ▶ para parar telnet

```
# systemctl stop telnet.socket
```

# ejemplos fedora systemctl

- ▶ ejemplos.
  - ▶ para habilitar telnet al iniciar el sistema  
`# systemctl enable telnet.socket`
  - ▶ para inhabilitar telnet al iniciar el sistema  
`# systemctl disable telnet.socket`
  - ▶ para iniciar el servicio sshd  
`# systemctl start sshd.service`
  - ▶ para inhabilitar sshd al iniciar el sistema  
`# systemctl disable sshd.service`
  - ▶ to ver que *units* hay disponibles  
`# systemctl list-units`

# inicio de servicios en fedora

- ▶ podemos añadir nuestros scripts al directorio `/etc/init.d` para hacer que inicien (o paren) servicios al arrancar
- ▶ nuestros scripts deben entender los parámetros `start`, `stop`, `status`, `restart` ...
- ▶ los servicios pueden ser entonces manejados con `ntsysv`, `chkconfig` o `system-config-services` y ser iniciados o detenidos con el comando `service`
- ▶ sin embargo, para que un script pueda ser manejado de esta manera, debemos suministrar una cabecera para `chkconfig` o incluir una cabecera LSB

# chkconfig

- ▶ `chkconfig` es una interfaz para el mantenimiento de `/etc/rcN.d`.  
Ejemplo de uso:

```
# chkconfig --level 5 pcmcia on
```

- ▶ `pcmcia` es el nombre del script.
- ▶ 5 es el runlevel que queremos afectar.
- ▶ `on` queremos que en el runlevel 5 se inicie
- ▶ crea un enlace en `/etc/rc5.d/S24pcmcia`
- ▶ el enlace es a `/etc/init.d/pcmcia`
- ▶ el número 24 lo obtiene de la línea comentario del script `#chkconfig: 2345 24 96`
  - ▶ script susceptible de aparecer en los runlevels 2345
  - ▶ para iniciarlo lo haría con el número 24 (`S24pcmcia`)
  - ▶ para pararlo con el número 96 (`K96pcmcia`)
- ▶ En fedora existe interfaz gráfica para el mantenimiento de estos enlaces (`system-config-services`)

# ejemplos de cabeceras

- ▶ ejemplo de cabecera para chkconfig,

```
# chkconfig: 2345 20 80
# description: Estas lineas apareceran como las descripcion \
#               del servicio proporcionado por este script.
```

- ▶ cabecera LSB

```
### BEGIN INIT INFO
# Provides: foo
# Required-Start: bar
# Default-Start: 2 3 4 5
# Default-Stop: 0 1 6
# Description: Foo init script
### END INIT INFO
```



# ejemplo de script trivial para ntsysv o chkconfig

```
#!/bin/sh

# chkconfig: 2345 20 80
# description: script que no hace nada
#

case "$1" in
start)
    echo starting..
    ;;
stop)
    echo stopping...
    ;;
status)
    echo ni idea
    ;;
```

# Arranque de Servicios

Proceso de arranque

runlevels

linux debian: `insserv`

fedora linux

Actividades

# Actividades

- ▶ Cambiar el runlevel al modo monousuario. Volver al runlevel normal
- ▶ Probar a iniciar y detener el servicio *sshd* (en debian y en fedora) y comprobar en cada caso que se hace correctamente
- ▶ Probar a habilitar y deshabilitar el servicio *sshd* en el inicio del sistema (en debian y en fedora) y comprobar en cada caso que se hace correctamente
- ▶ Incluir el script trivial suministrado como ejemplo en el sistema de inicio de fedora y comprobar que se maneja con el comando *service*
- ▶ Modificar el escript *backup.sh* realizado anteriormente, de manera admita los parámetros *start stop*. Integrarlo en el sistema de manera que haga copias de seguridad al encender la maquina y al apagarla envíe un mail a los usuarios comunicándoles la fecha de su última copia de seguridad

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Archivos de Log

syslog

logrotate

Actividades

# Archivos de Log

- ▶ Un archivo de log guarda información acerca del funcionamiento de alguna parte del sistema.
- ▶ En linux los archivos de log se encuentran en `/var/log`.
- ▶ El programa que se encarga de almacenar la información en los archivos de log se denomina `syslog`. Este programa está corriendo en segundo plano en todo momento.
- ▶ En linux es usual que esté instalado un “sustituto” de `syslog`: *`rsyslog`*, *`syslog-ng`* ...
- ▶ De aquí en adelante nos referiremos genericamente a *`syslog`*

# Archivos de Log

- ▶ Los mensajes almacenados por syslog proporcionan la siguiente información:
  - ▶ Fecha y Hora del mensaje.
  - ▶ Nombre de la Máquina donde se produjo.
  - ▶ Servicio que ha producido el mensaje.
  - ▶ Texto descriptivo.

# Archivos de Log

- ▶ Syslog puede configurarse para que decida que hacer con las entradas de log que le mandan las aplicaciones, en función de que aplicación ha generado el mensaje, y su importancia.
- ▶ El mensaje puede:
  - ▶ Descartarse.
  - ▶ Escribir el mensaje en un fichero de log. Este fichero puede ser exclusivo para esa aplicación, o un fichero donde se almacenan los logs de varias aplicaciones.
  - ▶ Enviar el mensaje a otra máquina de la red que corra syslog. De esta forma se pueden centralizar los archivos de log en una red.
  - ▶ Enviar el mensaje a todos los usuarios conectados a la máquina.
  - ▶ Escribir el mensaje en la consola del sistema.



# Configuración de Syslog

- El fichero de configuración de syslog es `/etc/syslog.conf`. Está formado por líneas con el siguiente formato:

```
Servicio.Prioridad Acción
```

```
Servicio1,Servicio2.Prioridad Acción
```

```
Serv1.Prio1;Serv2.Prio2 Acción1 Acción2
```

# Configuración de Syslog: Servicios

- ▶ Los servicios que se pueden definir son los siguientes:
  - ▶ authpriv: Autenticación de usuarios.
  - ▶ cron: Tareas planificadas.
  - ▶ daemon: Todos los demonios que no tienen nombre de servicio propio.
  - ▶ ftp: Servidor de Ftp.
  - ▶ kern: Mensajes del Kernel.
  - ▶ lpr: Mensajes del gestor de colas de impresión.
  - ▶ mail: Mensajes del servidor de correo.
  - ▶ mark: Marcas de tiempo.
  - ▶ news: Servidor de noticias.
  - ▶ syslog: Mensajes producidos por el servidor de syslog.
  - ▶ user: Mensajes producidos por aplicaciones de usuario.
- ▶ Si se utiliza un \* como servicio se entiende que se desea interceptar todos los mensajes.

# Configuración de Syslog: Prioridades

- ▶ Las prioridades de los mensajes son:
  - ▶ emerg.
  - ▶ alert.
  - ▶ crit.
  - ▶ err.
  - ▶ warning.
  - ▶ notice.
  - ▶ info.
  - ▶ debug.
  - ▶ none.
- ▶ Un \* indica todos los niveles de prioridad.

# Configuración de Syslog: Acciones

- ▶ Si el mensaje encaja con la descripción de servicio y prioridad, se pueden realizar distintas acciones con ellos.
  - ▶ Ficheros del sistema: El mensaje se escribe en un fichero. El fichero debe ser especificado con la ruta completa. (Debe empezar por /).
  - ▶ Máquina Remota: Para enviar el mensaje a otra máquina, se pone el nombre de ésta precedido con una @.
  - ▶ Enviar a un determinado grupo de usuarios: Se indican los usuarios poniendo sus logins separados por ,.
  - ▶ Todos los usuarios. Se indican con un \*.

# Configuración de Syslog: Ejemplo

► Ejemplo de syslog.conf:

```
auth,authpriv.*          @aula01
*.*;auth,authpriv.none   /var/log/syslog
daemon.*                 /var/log/dmn.log
kern.*                    root
lpr.*                     /var/log/lpr.log
mail.*                    /var/log/mail.log
user.*                     /var/log/user.log
uucp.*                     /var/log/uucp.log
```

# Archivos de Log

syslog

logrotate

Actividades

# Rotación de archivos de log

- ▶ Uno de los mayores problemas con los archivos de log es que tienden a crecer mucho con el tiempo.
- ▶ Un archivo grande es difícil de manejar. Además, es deseable ir borrando las entradas despues de un cierto tiempo.
- ▶ En algunas distribuciones los archivos de log generados por syslog son rotados directamente por `/etc/cron.daily/sysklogd` y `/etc/cron.weekly/sysklogd`. En otras dsitribuciones todos los archivos de log son rotados por logrotate.

# Rotación de archivos de log generados por syslog

- ▶ Todos los archivos de log de \*.\* (todos los servicios y todos los niveles de prioridad) generados por syslog son rotados directamente por `/etc/cron.daily/sysklogd`.  
`/usr/sbin/syslogd-listfiles` informa de cuales son.
- ▶ El resto de los archivos de log generados por syslog son rotados semanalmente por `/etc/cron.weekly/sysklogd`.  
`/usr/sbin/syslogd-listfiles --weekly` informa de cuales son.



# logrotate

- ▶ Logrotate es un programa que cada cierto tiempo renombra los archivos de log y borra los más viejos.
- ▶ Puede especificársele qué archivos se rotan y cómo
- ▶ El fichero de configuración reside en `/etc/logrotate.conf`:

# logrotate: fichero de configuración

```
# rotar semanalmente. Podría ser monthly o daily
weekly

# mantener 4 copias atrasadas
rotate 4

# crear un fichero de log vacío tras hacer la rotación
create

# comprimir los ficheros
compress

# No comprimir el fichero de log la primera vez.
delaycompress

# Directorio cuyos ficheros se incluyen
include /etc/logrotate.d

# Configuración específica para un fichero
/var/log/wtmp {
    missingok # Si no existe no pasa nada
    monthly
    rotate 1
}
```

# Archivos de Log

syslog

logrotate

Actividades

# Actividades

- ▶ Determinar cual de los sustitutos de *syslog* tenemos en nuestro sistema
- ▶ Cambiar la configuración del sistema, para que envíe los mensajes del kernel a nuestro usuario, en vez de a un fichero de log
- ▶ Probar a enviar los mensajes de autenticación a otra máquina.
- ▶ Comprobar desde donde se invoca a *logrotate*
- ▶ Crear una entrada en *logrotate* para un archivo de log definido por nosotros en */etc/syslog.conf* (o su análogo) y comprobar como se produce la rotación.

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

**Sistemas de Ficheros**

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Sistemas de Ficheros

## Sistemas de Ficheros

Control de Uso de Disco: Quota

Actividades

NFS: Network File System

SSHFS: Secure Shell File System

Actividades

# Particiones y fstab

- ▶ Es habitual en un sistema Unix/Linux que el sistema esté instalado en más de una partición.
- ▶ Se suelen crear particiones aparte para varios de los directorios del sistema, para evitar que un fallo que llene una de ellas afecte a las demás.
- ▶ En general se suelen poner en particiones aparte:
  - ▶ `/tmp`: Este es el directorio donde las aplicaciones ponen sus ficheros temporales. Es, por tanto, una partición susceptible de ser llenada por un fallo en alguna aplicación, o por un usuario malicioso.

# Particiones y fstab II

- ▶ ▶ /var: En este directorio se encuentran los logs del sistema. Que sea posible escribir en los logs es muy importante, más aun en la situación anómala de que el disco esté lleno, porque puede indicar un ataque local.
- ▶ /home: Al utilizar una partición para /home es posible usarla con más de una distribución. Además, al ser un directorio donde puede escribir cualquier usuario, es susceptible de llenarse sin previo aviso.
- ▶ /usr: En usr se encuentra instalado el grueso del espacio de disco de los programas. Es por tanto un directorio que necesita bastante espacio en disco.
- ▶ /boot: En ella colocamos típicamente los ficheros de arranque del sistema.



# Particiones y fstab III

- ▶ En el fichero `/etc/fstab` se incluye información acerca de los sistemas de ficheros que el sistema puede montar.
- ▶ Utilizando este fichero, es posible configurar para que determinadas particiones se monten al arrancar el sistema, dar opciones de montaje por defecto, dar permiso a los usuarios para montarlas...
- ▶ Ejemplo:

```
# <file system> <mount point> <type> <options> <dump> <pass>
/dev/hda1 / xfs defaults 0 1
/dev/hda7 none swap sw 0 0
/dev/hda5 /var xfs defaults 0 1
/dev/hda6 /usr xfs defaults 0 1
/dev/hda2 /home xfs defaults 0 1
/dev/hda3 /tmp xfs defaults 0 1
proc /proc proc defaults 0 0
/dev/cdrom /cdrom iso9660 defaults,ro,user,noauto 0 0
```

# Particiones y fstab IV

- ▶ Los campos de `/etc/fstab` son los siguientes:
  - ▶ Partición: indica la partición que se va a montar. Hay sistemas de ficheros especiales, como `/proc`, que no tienen asociado ninguna partición.
  - ▶ Punto de montaje: indica el directorio donde esa partición va a ser montada por defecto.
  - ▶ Sistema de ficheros: Es el tipo de sistema de ficheros con el que ha sido formateada la partición.
  - ▶ Opciones: Cualquier otra opción de montaje que queramos darle. Algunas usuales son `ro`, para sistemas de ficheros de solo lectura; `user`, para que cualquier usuario pueda montarla; o `noauto`, para que no se monte automáticamente cuando arranca el sistema; `noexec` no se ejecutar nada de esa partición o `nosuid`, la partición no admite ficheros `setuid`. Existen además opciones específicas del tipo de sistemas de ficheros.

# Particiones y fstab V

- ▶ Dump: 1 o 0 según si la partición va a tener un backup controlado por dump o no.
- ▶ Pass: Si este campo está a 1, la partición es susceptible de ser verificada al arrancar el sistema si se supera un cierto número de montajes.

# Creación de Sistemas de Ficheros

- ▶ Para poder utilizar una partición es necesario darle antes un determinado formato. Este formato sirve para crear las estructuras en disco necesarias para almacenar la información.
- ▶ En linux se pueden utilizar varios sistemas de ficheros que pueden ser utilizados: ext2, ext3, reiserfs, xfs y jfs.
- ▶ Los sistemas de ficheros se crean utilizando *mkfs -t tipo*. También se puede utilizar la utilidad respectiva de cada tipo de sistema de ficheros, *mkfs.tipo* (Ej: *mkfs.ext2*, *mkfs.ext3* ...).

```
mkfs.ext2 /dev/hda2
```

# Mantenimiento de Sistemas de Ficheros

- ▶ Periodicamente es recomendable comprobar que el sistema de ficheros se encuentra en un estado coherente.
- ▶ Cada sistema de ficheros acostumbra a traer una utilidad para comprobar que el contenido de una partición de este tipo es correcto, y corregirlo en el caso contrario.
- ▶ Por ejemplo, para ext2 y ext3 esta utilidad se llama *e2fsck*.
- ▶ Los sistemas de ficheros transaccionales (Todos menos ext2) son menos susceptibles a este tipo de problemas. Esto se debe a que modifican los metadatos utilizando un sistema de transacciones similar al de una base de datos.

# Acceso a Sistemas de Ficheros

- ▶ Para acceder al contenido de un sistema de ficheros, primero ha de ser montado. Al montar un sistema de ficheros, su contenido se sitúa debajo de un directorio del árbol.
- ▶ Para montar una partición, se utiliza el comando mount:

```
mount -t tipofs -o opciones dispositivo punto-montaje  
mount -t ext2 /dev/hda /mnt/prueba
```

- ▶ Normalmente no es necesario indicar el tipo del sistema de ficheros.
- ▶ Las opciones más comunes son:
  - ▶ ro: El sistema de ficheros se montará en modo de solo lectura.
  - ▶ rw: Se puede escribir en el sistema de ficheros.
  - ▶ loop: Permite montar un fichero normal que contenga un sistema de ficheros en su interior.
  - ▶ sync: El sistema de ficheros se sincroniza con disco de forma inmediata tras una escritura.

# Sistemas de Ficheros

Sistemas de Ficheros

Control de Uso de Disco: Quota

Actividades

NFS: Network File System

SSHFS: Secure Shell File System

Actividades

# Quota

- ▶ Quota permite establecer límites en el uso de disco a los usuarios, tanto en el espacio ocupado, como en el número de ficheros que puede tener. Se puede configurar el sistema para que permita superar el límite durante periodos limitados de tiempo.
- ▶ La configuración de quota es por usuario (y/o grupo) y sistema de ficheros.
- ▶ Para utilizarlo es necesario que el kernel tenga soporte para quota y tener instalado los paquetes correspondientes (ej. en debian `sarge quotay quotatool`)
- ▶ Los sistemas de ficheros en los que queremos habilitar quota deben ser montados con la opción `usrquota` y/o `grpquota`.
- ▶ Las quotas se almacenan en unos archivos `quota.user` y `quota.group` en el directorio raíz del sistema de archivos donde se establece la quota.



# Quota

Para el manejo de las cuotas tenemos las siguientes utilidades:

- ▶ *quotacheck*. Crea, comprueba y/o repara los archivos de quota de un sistema de ficheros.
- ▶ *quotaon* y *quotaoff* activan y desactivan respectivamente el funcionamiento de las cuotas en un sistema de ficheros.
- ▶ *edquota* Permite modificar las cuotas.
- ▶ *repquota* o *quota* Informa del estado de las cuotas.

# Quota: Aplicación de cuotas

Ejemplo: Pasos necesarios para usar de cuotas de usuario en un sistema de ficheros

- ▶ Debe haber soporte para cuotas en el kernel.
- ▶ El sistema de ficheros debe haber sido montado con la opción `usrquota`.

```
#mount -t ext2 -o usrquota /dev/hda2 /home
```

- ▶ Debe crearse el fichero de cuotas en el directorio raíz de la partición en la que queremos cuotas.

```
#quotacheck -uv /dev/hda2
```

- ▶ Debe activarse la quota para ese sistema de ficheros con `quotaon`.

```
#quotaon /dev/hda2
```

# Quota: Aplicación de cuotas II

- ▶ Deben establecerse las cuotas para los usuarios que queramos que tengan quota:
  - ▶ *edquota -u nombre*. Abre el editor definido en \$EDITOR en el que se pueden fijar los límites soft y hard para inodos y bloques. El límite soft puede ser superado durante un cierto tiempo. Este tiempo se denomina tiempo de gracia.
  - ▶ *edquota -p prototipo nombre*. Establece las cuotas del usuario nombre iguales a las de prototipo.
  - ▶ *edquota -t*. Permite fijar el tiempo de gracia.

# Quota: Aplicación de cuotas III

- ▶ Puede comprobarse el estado las cuotas con:
  - ▶ *repquota particion* Muestra información sobre el uso de cuotas en esa partición.  
`#repquota /dev/hda2`
  - ▶ *quota user* Muestra el estado de las cuotas de un usuario.  
`#quota -v alumno`

# Sistemas de Ficheros

Sistemas de Ficheros

Control de Uso de Disco: Quota

**Actividades**

NFS: Network File System

SSHFS: Secure Shell File System

Actividades

# Quota: Ejercicios

- ▶ Ejercicios:
  - ▶ Fijar las cuotas para un usuario normal que tengamos creado en la máquina.
  - ▶ Comprobar como no nos deja pasar del límite hard, tanto de inodos como de bloques.
  - ▶ Cambiar el periodo de gracia.

# Sistemas de Ficheros

Sistemas de Ficheros

Control de Uso de Disco: Quota

Actividades

**NFS: Network File System**

SSHFS: Secure Shell File System

Actividades

# NFS I

- ▶ NFS: Network File System.
- ▶ Es un protocolo de red desarrollado por Sun para permitir montar particiones en máquinas remotas como si estuvieran en un disco local.
- ▶ El servidor especifica en su configuración que partes de su árbol de directorios desea hacer accesible y a quien.
- ▶ En clusters se utiliza habitualmente para poder utilizar en todas las máquinas los directorios de los usuarios.



# NFS II

- ▶ Para que funcione el servidor nfs es necesario:
  - ▶ Tener activada la opción de servidor nfs en el kernel (en caso de no tenerla recompilar y seleccionar la opción File Systems -> Network File Systems -> NFS Server Support, y File Systems -> Network File Systems -> Provide NFSv3 support)
  - ▶ Tener instaladas las paquetes correspondientes según la distribución (ej. en debian sarge nfs-common y nfs-kernel-server o nfs-user-server)
- ▶ Para que funcione el cliente nfs:
  - ▶ Tener activado el soporte para cliente nfs en el kernel. (La opción necesaria es File Systems -> Network File Systems -> NFS file system Support, y File Systems -> Network File Systems -> Provide NFSv3 client support.)

# NFS: Configuración Servidor

- ▶ La configuración del servidor NFS se almacena en `/etc/exports`.
- ▶ Cada línea indica un directorio local a exportar y como se exporta.  
`directorio cliente1(opciones) cliente2(opciones) ...}`
- ▶ Los clientes se pueden especificar por:
  - ▶ **Nombre DNS:** Se admiten wildcards para los nombres.
  - ▶ **Dirección IP:** Se pueden utilizar rangos, indicando la máscara o el número de bits.
  - ▶ **Netgroup NIS:** Se pueden especificar grupos NIS como `@group`.

# NFS: Configuración Servidor II

- ▶ Opciones principales de montaje:
  - ▶ **ro:** El directorio se exporta en modo de solo lectura (Por defecto).
  - ▶ **rw:** El directorio se exporta en modo de lectura-escritura.
  - ▶ **sync:** Las operaciones de escritura no se completan hasta que el servidor ha escrito los datos en almacenamiento persistente.
  - ▶ **async:** Las operaciones de escritura se completan antes de que todos los datos se hayan escrito en almacenamiento persistente.
- ▶ Una vez configurado el servicio puede iniciarse y/o pararse con el script correspondiente en `/etc/init.d`.

# NFS: Ejemplo de /etc/exports

```
# Fichero de ejemplo para /etc/exports.  
/home  pc*.cesga.es(rw,async) 192.144.23.3(ro)  
/var/lib 192.144.23.0/24(rw) 192.168.1.0/255.255.255.0(rw)  
/etc  @cesganis(rw)  
/pub  (ro)
```

# NFS: Configuración Cliente

- ▶ Para montar una partición nfs desde el cliente, utilizaremos `mount`  
`mount maquina:/share /directorio_local`
- ▶ Ejemplo: `mount pc0.cesga.es:/home /mnt`
- ▶ Para que se monten cada vez que el sistema se inicia, hay que añadirlos en `/etc/fstab`.
- ▶ Ej: `maquina:/share /mnt nfs defaults 0 0`

# Sistemas de Ficheros

Sistemas de Ficheros

Control de Uso de Disco: Quota

Actividades

NFS: Network File System

**SSHFS: Secure Shell File System**

Actividades

# Secure Shell File System

- ▶ Permite acceder a ficheros en un sistema donde tenemos acceso por `ssh` como si estuviesen en el sistema local
- ▶ No es necesario tener privilegios de administrador
- ▶ Toda comunicación circula por un canal seguro
  - ▶ Necesita soporte FUSE (Filesystem in User Space) en el kernel de la máquina local
  - ▶ El usuario debe pertenecer al grupo `fuse` en la máquina local
  - ▶ El usuario debe tener acceso `ssh` a la máquina remota

# Secure Shell File System

- ▶ Para *montar* un directorio remoto

```
sshfs user@maquina_remota:dir_remoto dir_local
```

- ▶ Para *desmontar* un directorio remoto ya montado

```
fusermount -u directorio_local
```



# Sistemas de Ficheros

Sistemas de Ficheros

Control de Uso de Disco: Quota

Actividades

NFS: Network File System

SSHFS: Secure Shell File System

Actividades

# NFS: Actividades

- ▶ Configurar NFS para que exporte el directorio /home a otro pc.
- ▶ Comprobar que se puede montar el directorio de otra máquina.
- ▶ Comprobar que no se puede montar el de otra máquina que no este en la lista permitida.
- ▶ Añadir una linea a fstab para que lo monte cada vez que se inicia el sistema.
- ▶ Montar con sshfs nuestro directorio HOME de otra máquina

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Firewall y Nat: Iptables

Firewall

Nat

Actividades

# Iptables/NetFilter I

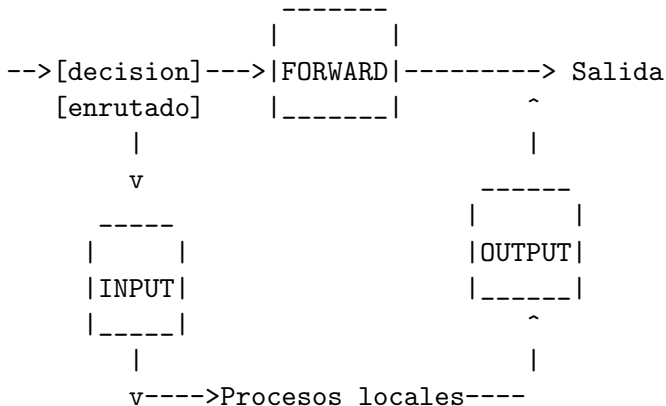
- ▶ Un filtro de paquetes es un programa que comprueba las cabeceras de los paquetes de red y decide realizar alguna opción o no sobre él.
- ▶ En Linux existe un filtro de paquetes que reside en el kernel como parte de la infraestructura de manipulación de paquetes del kernel (Netfilter).
- ▶ Al residir en el kernel, la configuración se pierde al reiniciar, por lo que hay que incluirla en algún script de inicio.
- ▶ Netfilter contiene varias tablas para las distintas funciones que soporta. La tabla para el filtrado de paquetes es filter.

# Iptables/NetFilter II

- ▶ La tabla filter está formada por **cadenas**. Por defecto están definidas tres (INPUT, OUTPUT y FORWARD), pero pueden definirse más.
- ▶ Cada cadena contiene una lista de **reglas**. Las reglas especifican acciones a tomar frente a paquetes que encajan en una cierta descripción.
- ▶ Las reglas se van consultando por orden hasta que el paquete encaja en la descripción de alguna de ellas, y ejecuta la acción correspondiente.
- ▶ Si no encaja con ninguna de las reglas, se consulta la política por defecto de la cadena.

# Iptables/NetFilter III

- Recorrido de los paquetes por las tablas.



# Iptables/Netfilter: Uso de iptables

- ▶ Manipulación de cadenas:
  - ▶ Crear una cadena: `iptables -N nombre`.
  - ▶ Borrar una cadena: `iptables -X nombre`.
  - ▶ Cambiar la política por defecto: `iptables -P cadena destino`. Los destinos más usuales para un paquete son DROP(descartarlo), ACCEPT(aceptarlo) y REJECT(descartarlo y mandar un mensaje ICMP al emisor).
  - ▶ Listar las reglas de una cadena: `iptables -L cadena`.
  - ▶ Borrar todas las reglas de una cadena: `iptables -F cadena`.



# Iptables/Netfilter: Reglas

- ▶ Manipulación de reglas:
  - ▶ Añadir una regla a una cadena: iptables -A *cadena especificación*.
  - ▶ Borrar una regla de una cadena: iptables -D *cadena número*.
  - ▶ Insertar una regla: iptables -I *cadena número especificación*.
  - ▶ Reemplazar una regla: iptables -R *cadena número especificación*.

# Iptables/Netfilter: Reglas II: Selección de paquetes

- ▶ Algunas de las condiciones que podemos especificar para seleccionar paquetes son:
  - ▶ Por protocolo: `-p protocol`. Se admiten tcp, udp, icmp o all.
    - ▶ Por puerto de origen: `--sport port`.
    - ▶ Por puerto de destino: `--dport port`.
    - ▶ Fragmentos de paquetes: `-f`.
    - ▶ Identificar si es un paquete de conexión: `--syn`.
    - ▶ Para icmp: `--icmp-type tipo`. Por ejemplo, ping.
  - ▶ Por dirección de origen: `-s dirección[/mascara]`.
  - ▶ Por dirección de destino: `-d dirección[/mascara]`.
  - ▶ Por interfaz de red de entrada: `-i interfaz`.
  - ▶ Por interfaz de red de salida: `-o iterfaz`.
  - ▶ Puede usarse `!` para negar: `-p ! tcp` cualquier protocolo que no sea tcp.

# Iptables/Netfilter: Reglas III: Acciones

- Se indica lo que debe hacer netfilter con el paquete con `-j acción`. *Las opciones posibles son:*

**DROP** *El paquete se descarta. (terminaría dando un error de time out en el origen)*

**REJECT** *El paquete se rechaza. (daría un error de conexión rechazada en el origen)*

**ACCEPT** *El paquete se acepta*

**LOG** *El paquete continua su recorrido por las reglas, pero se guarda una entrada en los archivos de log.*

# Iptables/Netfilter: Ejemplos

- ▶ No permitir que ninguna máquina externa pueda conectarse al servidor web.

```
iptables -A INPUT -p tcp --destination-port 80 -j DROP
o
iptables -A INPUT -p tcp --destination-port 80 -j REJECT
```

- ▶ No aceptar pings.

```
iptables -A INPUT -p icmp --icmp-type ping -j DROP
```

- ▶ Especificar la política por defecto para los paquetes de entrada a rechazar.

```
iptables -P INPUT DROP
```

- ▶ Admitir las conexiones ya establecidas desde la máquina con destino la dirección \$DIR en eth0

```
iptables -A INPUT -i eth0 -p tcp -d $DIR -m state --state ESTABLISHED -j ACCEPT
```

# Iptables/Netfilter: Pruebas

- ▶ Crear una serie de comandos iptables que solo permitan el acceso al puerto ssh a la máquina contigua y a la nuestra propia.
- ▶ Crear una nueva cadena que haga log de los paquetes que se le manden. Añadir las reglas necerias para hacer log de todos los pings que vengan a nuestra máquina.
- ▶ Crear una regla que no permita conexiones desde la red 192.168.0.0/24, salvo para una única máquina de ese rango.

# Firewall y Nat: Iptables

Firewall

Nat

Actividades

# Nat I

- ▶ Nat es una técnica que consiste en cambiar las direcciones de origen y/o destino de un paquete. Si la máquina a la que se dirigía el paquete contesta, se realiza el cambio inverso cuando el paquete vuelve a atravesar el firewall.
- ▶ Nat tiene varias aplicaciones:
  - ▶ Dar conexión a más de una máquina con una sola dirección ip pública.
  - ▶ Utilizar una única máquina como interfaz con la red, y hacer que esta redirija los paquetes a los servidores que se encuentran detrás de ésta.
  - ▶ Implementar un proxy transparente. Los paquetes pueden redirigirse a una aplicación local que se encargue de hacer de proxy para el exterior sin necesidad de reconfigurar las máquinas.

# Nat II

- ▶ El Nat se puede dividir en dos tipos:
  - ▶ Nat de Origen: Se cambia la dirección de origen del paquete. Siempre se hace justo antes de enviar el paquete (POSTROUTING). Con este tipo de nat se puede dar conexión a más de una máquina con una única ip, por ejemplo.
  - ▶ Nat de Destino: Se cambia la dirección de destino del paquete. Siempre se hace justo después de recibir el paquete (PREROUTING). Con este tipo de nat se puede implementar un proxy transparente, por ejemplo.



## Nat III: Selección de Paquetes

- ▶ Para realizar Nat en linux se utiliza la tabla nat de netfilter. Esta tabla tiene dos cadenas definidas por defecto, que son PREROUTING y POSTROUTING.
- ▶ Para seleccionar una tabla sobre la que opera netfilter, se utiliza la opción -t de iptables: `iptables -t nat`.
- ▶ Los comandos para seleccionar paquetes son las mismas que para el filtrado de paquetes.

## Nat IV: Acciones sobre los paquetes

### ► Nat de Origen:

- Se utiliza la acción SNAT. El destino se indica con `-to dirección`. La dirección puede incluir puertos de destino (Para tcp y udp) y rangos.

Ej: `iptables -t nat -A POSTROUTING -j SNAT --to 1.2.3.4`

- Un caso especial de Nat de Origen es el MASQUERADING, que asigna automáticamente la ip del interfaz por el que sale el paquete. Es útil si la ip puede cambiar, como en una conexión por modem.

### ► Nat de Destino:

- Se utiliza la acción DNAT. La opción `-to dirección` se utiliza para cambiar la dirección de destino. Admite las mismas posibilidades que para snat.

# Firewall y Nat: Iptables

Firewall

Nat

Actividades

# Nat: Actividades

- ▶ Mandar todas las conexiones entrantes al servicio ssh a otra máquina de la red.
- ▶ Hacer masquerading para otra máquina de la red.

nota Para que se realice el *forwarding* de paquetes y por tanto poder hacer NAT hay que editar `/etc/sysctl.conf` (`net.ipv4.ip_forward=1`) o bien habilitarlo con `sysctl` o en `/proc`

```
# sysctl net.ipv4.ip_forward=1
o
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

Apéndice II: Instalación de software en linux

# Creación de un kernel personalizado

## Introducción

Obtención de los fuentes del kernel

Configuración del kernel

Compilación del kernel

Instalación del kernel

Instalación del kernel: simplificaciones

# Configuración del Kernel

## Motivos para cambiar el kernel en un sistema

- ▶ Queremos soporte para un dispositivo u opción que el kernel actual no tiene.
- ▶ Queremos eliminar algunas opciones del kernel por diversos motivos
  - ▶ no queremos que estén disponibles
  - ▶ queremos que el tiempo de carga sea menor
  - ▶ queremos que ocupe menos memoria

# Configuración del Kernel

Metodo más sencillo (aunque no siempre posible)

- ▶ Cargar o eliminar módulos *modprobe rmmmod ...*
- ▶ Añadir módulos de una *tercera parte*
- ▶ Instalar un nuevo kernel disponible en nuestra distribución
  - ▶ Ejemplo  

```
apt-get install linux-image-2.6.24-etchnhalf.1-686
```
- ▶ Si por cualquier motivo el método más sencillo no nos sirve, será necesario recompilar kernel



# Recompilación del kernel

Son necesarios los siguientes pasos

1. Obtención de los fuentes, bien del repositorio *kernel.org* o desde un paquete de nuestra distribución
2. Configuración de las opciones que queremos (`make config`, `make menuconfig` o `make xconfig`, según nuestra preferencia)
3. Compilación de kernel y los módulos
4. Instalación del kernel y los módulos

# Creación de un kernel personalizado

Intruducción

Obtención de los fuentes del kernel

Configuración del kernel

Compilación del kernel

Instalación del kernel

Instalación del kernel: simplificaciones

# Obtención de los fuentes del kernel

## ► Método 1

- El kernel se puede obtener en <http://www.kernel.org> o a través de cualquiera de sus réplicas
  - El fichero con el código de Linux comprimido tendrá extensión `.tar.gz` o `.tar.bz2`.
  - Se coloca en `/usr/src/`
  - Se descomprimen con `tar xvfz fichero.gz` y `tar xvjf fichero.bz2`
  - Se crea un enlace `/usr/src/linux` al directorio con los fuentes descomprimidos
  - **ventajas:** Podemos obtener cualquier versión del kernel incluida la más reciente

# Obtención de los fuentes del kernel

## ► Método 2

- Uso el paquete de fuentes del kernel de mi distribución
  - Ejemplo, en debian `apt-get install linux-source-2.6.24`
  - **ventajas:** Más cómodo, y el kernel ya tiene aplicados los *patches* de mi distribución. Además instala paquetes adicionales necesarios para la compilación

# Creación de un kernel personalizado

Intruducción

Obtención de los fuentes del kernel

**Configuración del kernel**

Compilación del kernel

Instalación del kernel

Instalación del kernel: simplificaciones

# Configuración del Kernel

- El kernel presenta una gran cantidad de opciones y drivers para compilar. Una forma obtener información del hardware de nuestro sistema es consultar la salida del comando `lspci`.

```
0000:00:00.0 Host bridge: Intel Corp. 82855PM Processor to I/O Controller
0000:00:01.0 PCI bridge: Intel Corp. 82855PM Processor to AGP Controller
0000:00:1d.0 USB Controller: Intel Corp. 82801DB (ICH4) USB UHCI #1 (rev 03)
0000:00:1d.1 USB Controller: Intel Corp. 82801DB (ICH4) USB UHCI #2 (rev 03)
0000:00:1d.7 USB Controller: Intel Corp. 82801DB (ICH4) USB2 EHCI (rev 03)
0000:00:1e.0 PCI bridge: Intel Corp. 82801BAM/CAM PCI Bridge (rev 83)
0000:00:1f.0 ISA bridge: Intel Corp. 82801DBM LPC Interface Controller
0000:00:1f.1 IDE interface: Intel Corp. 82801DBM (ICH4) Ultra ATA
0000:00:1f.5 Multimedia audio controller: Intel Corp. 82801DB (ICH4) AC'97
0000:00:1f.6 Modem: Intel Corp. 82801DB (ICH4) AC'97 Modem Controller
0000:01:00.0 VGA compatible controller: ATI Technologies Inc Radeon R250 Lf
0000:02:02.0 Ethernet controller: NetXtreme BCM5705 Gigabit Ethernet
0000:02:04.0 Network controller: Intel Corp. PRO/Wireless LAN 2100 3B
```

# Configuración del Kernel

- En el directorio `proc` también hay información; por ejemplo, el fichero `/proc/cpuinfo` contiene información acerca del procesador:

```
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 9
model name     : Intel(R) Pentium(R) M processor 1400MHz
stepping       : 5
cpu MHz        : 597.664
cache size     : 1024 KB
fdiv_bug       : no
hlt_bug        : no
f00f_bug       : no
coma_bug       : no
fpu            : yes
fpu_exception  : yes
cpuid level    : 2
wp             : yes
flags          : fpu vme de pse tsc msr mce cx8 sep mtrr pge mca cmov pat clflush dts acpi mmx fxsr sse
bogomips       : 1179.64
```

# Configuración del kernel

- ▶ Configurar el kernel consiste en editar un fichero de texto con las opciones que queremos que tenga el nuevo kernel
- ▶ Las opciones pueden compilarse directamente en el kernel, o dejarse como módulos, que podrán ser cargados más adelante cuando sean necesarios. De esta forma no se desperdicia memoria.
- ▶ Si deseamos utilizar alguna configuración anterior solo tenemos que copiar su fichero de configuración a `.config` dentro del directorio de los fuentes del kernel (`/usr/src/linux`). Si es de otra versión podemos importarlo con `make oldconfig`



# Configuración del kernel

- ▶ Dependiendo del método de configuración del kernel que utilicemos puede ser necesario tener instalados distintos paquetes, como las librerías de desarrollo de curses (ncurses-devel), el compilador gcc, el compilador g++, (libqt-mt-dev)...
- ▶ tres metodos de configuración
  - ▶ config
  - ▶ menuconfig
  - ▶ xconfig

# Configuración del kernel: make config

- ▶ Nos colocamos en el directorio de los fuentes del kernel  
`/usr/src/linux`
- ▶ Hacemos `make config`
- ▶ Se nos presentarán **UNA POR UNA** todas las opciones de configuración

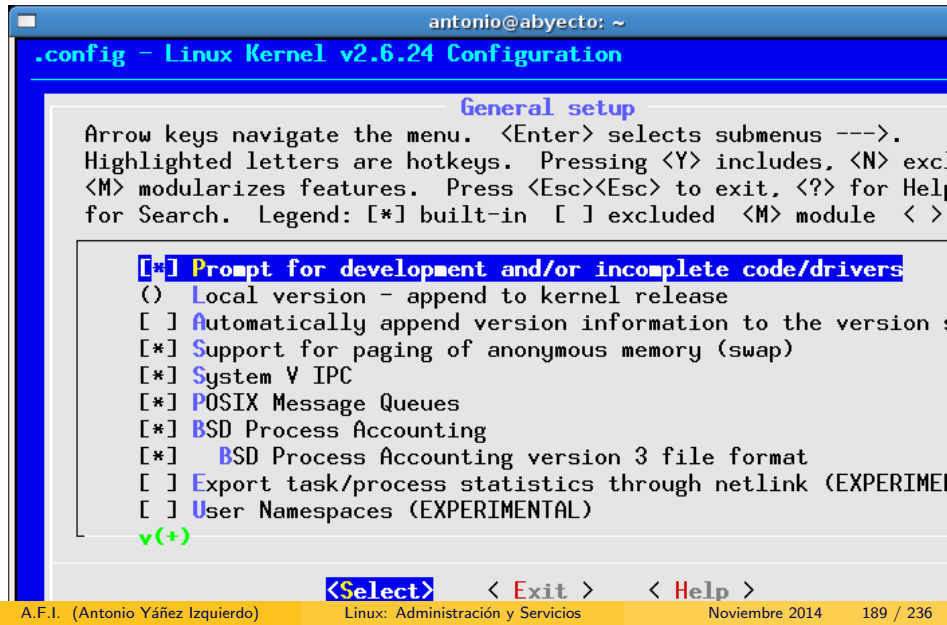
# Configuración del kernel: make config

```
antonio@abyecto: ~  
abyecto:/usr/src/linux# make config  
scripts/kconfig/conf arch/x86/Kconfig  
drivers/net/arm/Kconfig:54:warning: 'select' used by config symbol 'I  
refers to undefined symbol 'IXP4XX_NPE'  
drivers/net/arm/Kconfig:55:warning: 'select' used by config symbol 'I  
refers to undefined symbol 'IXP4XX_QMGR'  
drivers/net/wan/Kconfig:340:warning: 'select' used by config symbol 'I  
refers to undefined symbol 'IXP4XX_NPE'  
drivers/net/wan/Kconfig:341:warning: 'select' used by config symbol 'I  
refers to undefined symbol 'IXP4XX_QMGR'  
*  
* Linux Kernel Configuration  
*  
*  
* General setup  
*  
Prompt for development and/or incomplete code/drivers (EXPERIMENTAL)   
Local version - append to kernel release (LOCALVERSION) [] n  
Automatically append version information to the version string (LOCAL  
0) [N/y/?] b  
Automatically append version information to the version string (LOCAL
```

# Configuración del kernel: make menuconfig

- ▶ Nos colocamos en el directorio de los fuentes del kernel  
`/usr/src/linux`
- ▶ Hacemos `make menuconfig`
- ▶ Se nos presentará una serie de menús con las opciones de configuración.
- ▶ es necesario tener instaladas las librerías `ncurses`

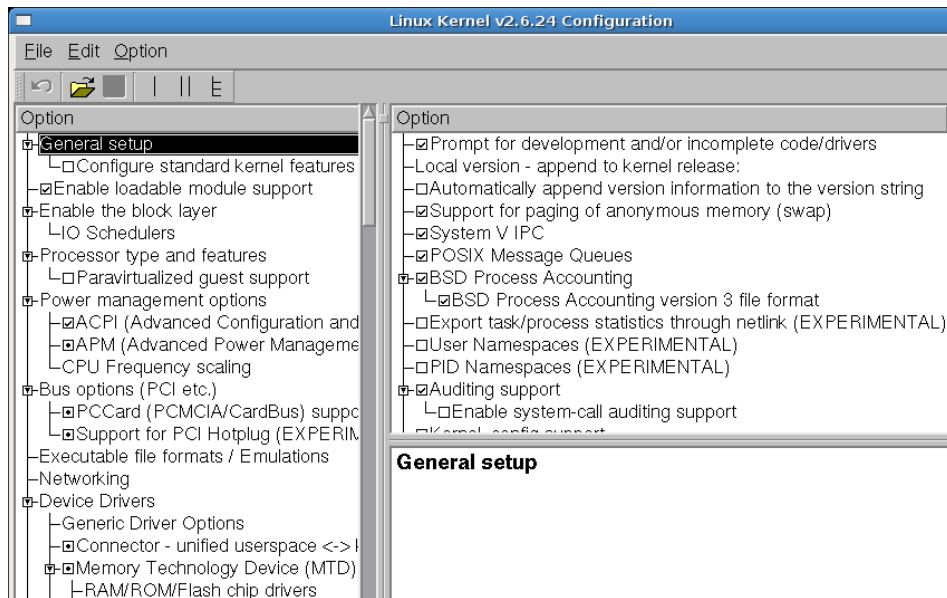
# Configuración del kernel: make menuconfig



# Configuración del kernel: make xconfig

- ▶ Nos colocamos en el directorio de los fuentes del kernel  
`/usr/src/linux`
- ▶ Hacemos `make xconfig`
- ▶ Se nos presentará un programa gráfico con las opciones de configuración.
- ▶ es necesario tener instaladas los paquetes `g++` y `libqt-mt-dev`
- ▶ Nótese que las opciones de configuración son las mismas por cualquiera de los métodos

# Configuración del kernel: make xconfig



# Creación de un kernel personalizado

Intruducción

Obtención de los fuentes del kernel

Configuración del kernel

**Compilación del kernel**

Instalación del kernel

Instalación del kernel: simplificaciones



# Compilación del kernel

- ▶ En el directorio `/usr/src/linux` editamos el fichero `Makefile` y añadimos algo en `EXTRAVERSION` que permitirá identificar el kernel que acabamos de compilar (en el ejemplo, el kernel se identificaría con **2.6.24-aula**)
- ▶ Hacemos `make clean` y `make dep` (no necesario en nuevas versiones del kernel)
- ▶ Hacemos `make bzImage` que compilará el kernel y colocará el fichero del kernel en `arch/i386/boot/bzImage` (si estamos en la arquitectura *i386*)
- ▶ Hacemos `make modules` para compilar los módulos

# Configuración del kernel: fichero makefile

```
antonio@abyecto: /usr/src/linux
VERSION = 2
PATCHLEVEL = 6
SUBLEVEL = 24
EXTRAVERSION = -aula
NAME = Arr Matey! A Hairy Bilge Rat!

# *DOCUMENTATION*
# To see a list of typical targets execute "make help"
# More info can be located in ./README
# Comments in this file are targeted only to the developer, do not
# expect to learn how to build the kernel reading this file.

# Do not:
# o use make's built-in rules and variables
#   (this increases performance and avoids hard-to-debug behaviour);
# o print "Entering directory ...";
MAKEFLAGS += -rR --no-print-directory

# We are using a recursive build, so we need to do a little thinking
# to get the ordering right.
#
```

# Creación de un kernel personalizado

Intruducción

Obtención de los fuentes del kernel

Configuración del kernel

Compilación del kernel

**Instalación del kernel**

Instalación del kernel: simplificaciones

# instalación del kernel

- El kernel, que está `arch/i386/boot/bzImage` (si estamos en la arquitectura *i386* lo copiamos al directorio `boot` (típicamente con el nombre `vmlinuz-version`)

```
#cp arch/i386/boot/bzImage /boot/kernel-2.6.24-aula
```

- Hacemos `make modules_install` para instalar los módulos. Se instalan en `/lib/modules/version-del-kernel` (`/lib/modules/2.6.24-aula` en nuestro caso)

# instalación del kernel

- ▶ El ramdisk de inicio podemos crearlo en caso necesario con *mkinitrd* o *mkinitramfs*

```
#mkinitramfs -o /boot/initrd.img-2.6.24-aula 2.6.24-aula
```

- ▶ Copiamos System.map al directorio boot

```
#cp System.map /boot/System.map-2.6.24-aula
```

- ▶ Guardamos la configuración por si queremos reutilizarla

```
#cp .config /boot/config-2.6.24-aula
```

# Instalación del kernel

- ▶ Habría que añadir el kernel que acabamos de compilar (y en su caso la imagen de inicio) al gestor de arranque (fichero `/boot/grub/menu.lst` en el caso de usar *grub* y fichero `/etc/lilo.conf` en el caso de usar *lilo*). *update-grub* en nuevas versiones *grub*
- ▶ Coviene dejar el kernel antiguo y así, en caso de haber algun problema, el sistema puede arrancar con el viejo kernel.

# Instalación del kernel

- ▶ El kernel puede compilarse en una máquina distinta de donde se va a instalar (por razones de espacio o velocidad). Lo que hay que hacer es, después de compilado
  - ▶ llevar a la nueva máquina el directorio de los módulos (completo) y colocarlo en `/lib/modules`
  - ▶ llevar a la nueva máquina el archivo del kernel, la imagen de arranque (*initrd.img-2.6.24-aula*) en caso de usarse, el archivo *System.map* y el archivo de configuración (si queremos conservarlo) y colocarlos en el directorio `/boot`
  - ▶ Incluir el nuevo kernel (y la imagen de arranque si procede) en el gestor de arranque de la máquina donde se quiere instalar

# Creación de un kernel personalizado

Intruducción

Obtención de los fuentes del kernel

Configuración del kernel

Compilación del kernel

Instalación del kernel

Instalación del kernel: simplificaciones



# Instalación del kernel: simplificaciones

- ▶ En algunas distribuciones el proceso de compilar e instalar el nuevo kernel puede simplificarse
- ▶ Lo que se hace es, una vez configurado el kernel, se genera un paquete que puede ser instalado con el sistema de paquetes de la distribución
- ▶ Ventaja adicional: si queremos llevarnos el kernel a otra máquina solo hay que llevar el paquete ya realizado
- ▶ En *debian* y *ubuntu* podemos usar `make-kpkg` para generar un paquete del kernel

```
# make-kpkg --append-to-version=-prueba  
kernel_image --initrd
```

# Instalación del kernel: simplificaciones

- ▶ El paquete, en el directorio `/usr/src` puede instalarse con `dpkg -i`  
`# dpkg -i kernel-image-mi_version.deb`
- ▶ Y si queremos que no sea susceptible de ser actualizado de manera automática ponemos el paquete en estado *hold*, lo que (en *debian* y *ubuntu*) podemos hacer con *dselect* o

```
# echo "kernel-image-mi_version hold" |  
dpkg --set-selections
```

# Gestores de Arranque

- ▶ La carga del kernel al principio del arranque del sistema la realiza un gestor de arranque. El gestor carga la imagen del kernel en memoria, la descomprime y comienza su ejecución.
- ▶ El gestor de arranque también es capaz de iniciar otros sistemas operativos, lo que es una ventaja de cara a tener más de un sistema operativo en la máquina.
- ▶ En linux se utilizan principalmente dos gestores de arranque:
  - ▶ Lilo, que es el más antiguo de los dos. Su principal desventaja con respecto a grub es que es necesario ejecutarlo con cada actualización del kernel.
  - ▶ Grub: Gestor de arranque que elimina la necesidad de ejecutarlo con cada actualización.

# Recompilación del kernel: resumen

## 1 Obtención de los fuentes.

- ▶ Dos posibilidades
  - ▶ Obtenerlos de *www.kernel.org*
  - ▶ Obtenerlos del repositorio de nuestra distribución. Por ejemplo en debian:  

```
#apt-get install linux-source-2.6
```
- ▶ Una vez colocados en `/usr/src` descomprimirlos y crear un enlace simbólico `/usr/src/linux` al directorio de los fuentes del kernel

```
host:/usr/src# tar xvjf linux-source-2.6.32.tar.bz2
```

```
host:/usr/src# ln -s linux-source-2.6.32 linux
```

# Recompilación del kernel: resumen (II)

## 2 Configuración del kernel

- ▶ La configuración reside en el archivo `/usr/include/linux/.config`
- ▶ Si queremos importar una configuración de una versión anterior del kernel: `make oldconfig`
- ▶ Tres métodos de configurar. Todos hacen lo mismo pero de distinta manera
  - ▶ `make config`
  - ▶ `make menuconfig`
  - ▶ `make xconfig`
- ▶ Después de configurar editar el fichero `/usr/include/linux/Makefile` y poner un identificador en `EXTRAVERSION` que identificará el nuevo kernel. Por ejemplo `EXTRAVERSION= -aula`

# Recompilación del kernel: resumen (III)

## 3 Compilación del kernel.

- ▶ Compilar el kernel

```
host:/usr/src/linux# make clean
```

```
host:/usr/src/linux# make dep
```

```
host:/usr/src/linux# make bzImage
```

- ▶ Compilar los módulos

```
host:/usr/src/linux# make modules
```

# Recompilación del kernel: resumen (IV)

## 4 Instalación del kernel

- ▶ instalar los módulos

```
host:/usr/src/linux# make modules_install
```

- ▶ Copiar los archivos `.config`, `System.map` y el kernel al directorio `/boot` con los nombres adecuados

```
host:/usr/src/linux# cp arch/x86/boot/bzImage /boot/vmlinuz-2.6.32-aula
```

```
host:/usr/src/linux# cp .config /boot/config-2.6.32-aula
```

```
host:/usr/src/linux# cp System.map /boot/System.map-2.6.32-aula
```

- ▶ Generar el archivo *initrd* mediante *mkinitrd* o *mkinitramfs*

```
host:/usr/src/linux# mkinitramfs -o /boot/initrd.img-2.6.32-aula 2.6.32-aula
```

# Recompilación del kernel: resumen (IVb)

## 4b Dar de alta el nuevo kernel en el gestor de arranque

- ▶ Si usamos el lilo editar `/etc/lilo.conf` y ejecutar `/sbin/lilo`
- ▶ Si usamos el grub (versiones anteriores a la 2) editar `/boot/grub/menu.lst`
- ▶ Si usamos el grub (versión 2 o posterior), ejecutar `update-grub2`
- ▶ Alternativamente a los pasos 3 y 4, podemos generar un paquete que pueda ser instalado con el sistema de paquetes de nuestra distribución. Por ejemplo en debian

```
host:/usr/src/linux# make-kpkg clean
host:/usr/src/linux# make-kpkg --append-to-version=-aula kernel_image --initrd
host:/usr/src/linux# cd /usr/src
host:/usr/src/linux# dpkg --install linux_image-2.6.32-aula_10.0Custom_i386.deb
```



Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

**Apéndice I: Configuración básica de la red en linux**

Apéndice II: Instalación de software en linux

# Apéndice I: Configuración básica de la red en linux

## Introducción

debian linux

fedora linux

# Configuración básica de ip v4

- ▶ los items a configurar son
  - ▶ el nombre de la máquina
  - ▶ cada tarjeta de red
  - ▶ las rutas
  - ▶ el dns (si se usa)

# Configuración básica de una tarjeta de red

- ▶ Los principales items que hay que configurar en una tarjeta de red
  - ▶ su dirección ip
  - ▶ la máscara de red (número de bits en su dirección ip que corresponden a la red)
  - ▶ dirección de broadcast

# Modos de configurar la red

- ▶ hay dos modos de configurar la red
  - ▶ **configuración manual:** se configuran manualmente todos los parámetros, bien directamente desde la línea de comando o bien en ficheros a través de los scripts de inicialización
  - ▶ **usando dhcp:** la tarjeta de red *pide su configuración* a una máquina en la red (the *el servidor dhcp*). Puede ser hecho directamente desde la línea de comando o bien en ficheros a través de los scripts de inicialización
- ▶ La mayor parte de los sistemas tienen también una utilidad gráfica que gestiona además la *wi-fi*. No vamos a tratar de esta utilidad ni de la configuración de redes *wi-fi*

# ifconfig

- ▶ el comando **ifconfig** configura tarjetas de red,
- ▶ usualmente localizado en **/sbin/ifconfig**
- ▶ puede configurar tanto manualmente como usando dhcp
- ▶ `ifconfig -a` muestra la configuración de las tarjetas de red

# configuración the dns

- ▶ reside en el fichero `/etc/resolv.conf`
- ▶ las opciones mas usuales en dicho fichero son
  - ▶ **nameserver** para especificar al dirección de un dns. Se admiten hasta 3
  - ▶ **domain** (opcional) para especificar el dominio local.
- ▶ ejemplo de fichero `/etc/resolv.conf`

```
domain dc.if.udc.es.  
nameserver 193.144.51.10  
nameserver 192.144.48.30
```

## fichero /etc/hosts

- ▶ este fichero contiene las relaciones nombre-dirección\_ip definidas localmente
- ▶ su formato es

```
ip_address      host_name      aliases
```

- ▶ ejemplo of /etc/hosts

```
127.0.0.1      localhost
192.168.1.99   abyecto.dc.fi.udc.es   abyecto
```



## fichero `/etc/nsswitch.conf`

- ▶ se usa para indicar que bases de datos de hosts, usuarios ... debe usar el sistema
- ▶ tambien especifica en que orden se deben usar
- ▶ en el siguiente ejemplo, para obtener las direcciones ip de las máquinas primero se consultan los ficheros locales y luego el *dns*

```
passwd:          compat
group:           compat
shadow:         compat

hosts:           files dns
networks:        files
```

# Apéndice I: Configuración básica de la red en linux

Introducción  
debian linux  
fedora linux

## debian: configuración de las taretas de red

- ▶ los nombres de las interfaces son *eth0*, *eth1*, *eth2* ...
- ▶ los nombres *eth0*, *eth1*, *eth2* ... se asignan **LA PRIMERA VEZ** que el kernel las encuentra. Esto se almacena en el fichero `/etc/udev/rules.d/70-persistent-net.rules`. Puede ser modificado en caso necesario
  - ▶ en versiones antiguas de linux el nombre se asignaba cada vez, y dependía del orden de carga de los *drivers*

# debian: configuración de las taretas de red

```
abyecto:/home/antonio# cat /etc/udev/rules.d/70-persistent-net.rules
# This file was automatically generated by the /lib/udev/write_net_rules
# program, run by the persistent-net-generator.rules rules file.
#
# You can modify it, as long as you keep each rule on a single
# line, and change only the value of the NAME= key.

# PCI device 0x11ab:0x4363 (sky2)
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}=="00:24:be:40:5c:4b",
ATTR{type}=="1", KERNEL=="eth*", NAME="eth0"

# PCI device 0x8086:0x4232 (iwlagn)
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}=="00:24:d6:0e:ae:a0",
ATTR{type}=="1", KERNEL=="wlan*", NAME="wlan0"
abyecto:/home/antonio#
```

# debian: configuración de las tarjetas de red

- ▶ **dhclient interface\_name** configura la tarjeta *interface\_name* mediante dhcp.
- ▶ **ifconfig interface\_name inet address addr netmask netmk broadcast bcast** configura la tarjeta *interface\_name* con la dirección *addr*, máscara *netmk* y dirección de broadcast *bcast*.

```
#ifconfig eth0 inet 192.168.1.100 netmask 255.255.255.0 broadcast 192.168.1.255
```

- ▶ **ifconfig interface\_name up** habilita la tarjeta

# debian: configuración de las tarjetas en el arranque

- ▶ el script que configura la red es (via `/etc/init.d/networking`)
- ▶ la configuración de las tarjetas reside en el fichero `/etc/network/interfaces` (página de manual `interfaces`)
- ▶ `/etc/hostname` Contiene el nombre del sistema

# debian: configuración de las tarjetas en el arranque

- Ejemplo de fichero `/etc/network/interfaces` con una sola tarjeta de red configurada

```
# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
#allow-hotplug eth0
auto eth0
iface eth0 inet static
address 192.168.1.99
netmask 255.255.255.0
network 192.168.1.0
broadcast 192.168.1.255
gateway 192.168.1.1
```

# debian: configuración de las tarjetas en el arranque

- Ejemplo de fichero `/etc/network/interfaces` con dos tarjetas de red

```
root@abyecto:~# cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo eth0 eth1
iface lo inet loopback

# The primary network interface
allow-hotplug eth0
iface eth0 inet dhcp
# internal network
allow-hotplug eth1
iface eth1 inet static
address 192.168.1.100
netmask 255.255.255.0
network 192.168.1.0
broadcast 192.168.1.255
```



# Apéndice I: Configuración básica de la red en linux

Introducción  
debian linux  
fedora linux

# fedora: configuración de las taretas de red

- ▶ En fedora las tarjetas se nombran de distinta manera
- ▶ Ya no se nombran *eth0*, *eth1*,...; obtienen nombres como *emN*, *empNpM*, *ensN*, *pNpM*
- ▶ Este nuevo sistema no depende ni del driver usado por la tarjeta ni de su MAC, los nombres dependen de cómo (donde) están conectadas, lo que hace más fácil su substitución

# fedora: configuración de las taretas de red

- Podemos usar los mismos comandos que en debian para configurar las interfaces
  - **dhclient interface\_name** configura la tarjeta *interface\_name* mediante dhcp.
  - **ifconfig interface\_name inet address addr netmask netmk broadcast bcast** configura la tarjeta *interface\_name* con la dirección *addr*, máscara *netmk* y dirección de broadcast *bcast*.

# fedora: configuración de las taretas de red

- ▶ *fedora linux* recomienda usar `ip addr` o `ip link` para configurar las interfaces
- ▶ Examples
  - ▶ `ip addr show` muestra la configuración de las interfaces
  - ▶ `ip link set p2p1 down` deshabilita *p2p1*
  - ▶ `ip link set p2p1 up` habilita *p2p1*
  - ▶ `ip addr add 192.168.2.100 dev p2p1` añade la dirección 192.168.2.100 a la tarjeta *p2p1*
  - ▶ `ip addr del 192.168.2.100 dev p2p1` elimina la dirección 192.168.2.100 de la interfaz *p2p1*
- ▶ con este comando puede asignarse mas de una dirección a una interfaz

## fedora: configuración de las tarjetas de red en el arranque

- ▶ si queremos que la s tarjetas de red se configuran en el arranque, debemos tener en cuenta los siguientes ficheros
- ▶ **/etc/sysconfig/network**. Un fichero que define las siguientes variables

NETWORKING=yes.or.no

HOSTNAME=fully.qualified.name

GATEWAY=ipaddr.of.the.gateway

GATEWAYDEV=interface

## fedora: configuración de las taretas de red en el arranque

- **/etc/sysconfig/network-scripts/ifcfg-interface\_name**. Un fichero para cada interfaz en el sistema. Define, entre otras, las siguientes variables

DEVICE=name

BOOTPROTO=protocol (none, bootp or dhcp)

IPADDR=address

NETMASK=mask

BROADCAST=address

Introducción

Instalación de linux

Shell Scripting

Tareas Planificadas

Arranque de Servicios

Archivos de Log

Sistemas de Ficheros

Firewall y Nat: Iptables

Creación de un kernel personalizado

Apéndice I: Configuración básica de la red en linux

**Apéndice II: Instalación de software en linux**

# paquetes de software en linux

- ▶ hay dos estándares principales para el manejo de paquetes de software en linux
  - ▶ **rpm** Introducido por RedHat (**R**edhat **P**ackage **M**anager). Es el estándar para RedHat, Fedora, Mandrake/Mandriva, Suse ...
  - ▶ **deb** Es el estándar en las distribuciones *debian* y derivados.
- ▶ On *ubuntu* y *debian* tenemos el sistema de paquetes **deb**
  - ▶ los ficheros están en el formato .deb
  - ▶ hay varias utilidades para tratar con los archivos(.deb): dpkg, apt-get, aptitude, synaptic ...



# sistema de paquetes deb

- ▶ los paquetes pueden residir en un repositorio centralizado o en un medio(CD, DVD ...)
- ▶ la localización de los paquetes se describe en `/etc/apt/sources-list`
- ▶ la mayor parte de la administración de paquetes se puede hacer con `apt-get`

`apt-get update`: actualiza la lista de paquetes disponible

`apt-get upgrade`: actualiza los paquetes a la versión mas reciente

`apt-get install package`: instala *package* en el sistema (junto con sus dependencias)

`apt-get remove package`: elimina *package* del sistema (junto con otros paquetes que dependan de él)

# sistema de paquetes deb

- hay otras utilidades para manejar los paquetes, todas ellas dependen de `/etc/apt/sources-list` para la localización de los paquetes

`aptitude` análogo a `apt-get` pero resuelve las dependencias de una manera un poco diferente

`dselect` utilidad controlada por menú

`dpkg` maneja los paquetes individualmente

`synaptic` utilidad gr<sup>o</sup>afica de debian (existen otras utilidades gráficas: *ubuntu's software center* ...)

# sistema de paquetes rpm

- ▶ fedora linux (al igual que redhat, suse y algunas otras distribuciones linux) usa el sistema de paquetes rpm
- ▶ hay un comando rpm (similar al dpkg de debian linux)
- ▶ la mayor parte de la administración de paquetes se hace con la utilidad yum (similar al apt-get de debian)

# fedora rpm packages

- la localización de los paquetes está descrita en el fichero `/etc/yum.repos.d` (o donde indique el fichero `/etc/yum.conf`)

`yum search` : busca en el repositorio

`yum install` : instala paquetes, junto con sus dependencias

`yum remove` : elimina un paquete del sistema (junto con otros paquetes que dependan de él)

`yum update` : actualiza paquetes

`yum clean` : limpia los ficheros caché del sistema de paquetes (por ejemplo para actualizar la lista de paquetes)

`yum localinstall` : instala un paquete disponible localmente